

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG ĐIỆN – ĐIỆN TỬ



ĐỒ ÁN TỐT NGHIỆP

Hệ thống Smart Building với Zigbee

Lê Đức Anh – 20224403

anh.ld224403@sis.hust.edu.vn

Chu Thiên Phú – 20224450

phu.ct224450@sis.hust.edu.vn

Ngành: Kỹ thuật Điện tử – Viễn thông

Chuyên ngành: Hệ thống nhúng thông minh và IoT

Giáo viên hướng dẫn: Phạm Thị Nhân

Chữ ký của GVHD

Hà Nội, 06/2026

LỜI CẢM ƠN

Mặc dù đã cố gắng hoàn thiện đồ án một cách tốt nhất, song do kiến thức và kinh nghiệm còn hạn chế nên không tránh khỏi những thiếu sót. Nhóm rất mong nhận được những ý kiến đóng góp quý báu từ các thầy cô để đồ án được hoàn thiện hơn.

Hà Nội, tháng 6 năm 2026

Nhóm sinh viên thực hiện

MỤC LỤC

DANH MỤC KÝ HIỆU VÀ CHỮ VIẾT TẮT	4
DANH SÁCH HÌNH VẼ	6
DANH SÁCH BẢNG	7
PHÂN CHIA CÔNG VIỆC	8
1 TỔNG QUAN ĐỀ TÀI	9
1.1 Bối cảnh và lý do chọn đề tài	9
1.2 Mục tiêu của đề án	10
1.2.1 Mục tiêu tổng quát	10
1.2.2 Mục tiêu cụ thể	10
1.3 Phạm vi và giới hạn	11
1.3.1 Phạm vi nghiên cứu và triển khai	11
1.3.2 Giới hạn của đề tài	11
1.4 Phương pháp thực hiện	11
1.5 Đóng góp chính của đề án	12
1.6 Cấu trúc báo cáo	13
2 CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ NỀN	14
2.1 Tổng quan IoT và Smart Building	14
2.2 Zigbee và IEEE 802.15.4	15
2.3 Vai trò thiết bị trong mạng Zigbee	16
2.4 Topology mạng Zigbee	16
2.5 Zigbee Protocol Stack	17
2.6 Zigbee Cluster Library và mô hình thiết bị	18
2.7 Gateway trong hệ thống IoT	19
2.8 MQTT và cơ chế Publish/Subscribe	19
2.9 UART, ASH và EZSP giữa Host và EFR32 NCP	20
2.10 REST API, Cloud Backend và Database	22
2.11 Mobile App và State Management	23
2.12 Firmware, Bootloader và OTA	23
2.13 Security trong Zigbee và IoT	24
2.14 Công nghệ sử dụng trong đề án	24
3 PHÂN TÍCH YÊU CẦU VÀ THIẾT KẾ HỆ THỐNG	26
3.1 Phạm vi và giả định thiết kế	26
3.2 Phân tích yêu cầu hệ thống	27
3.2.1 Tác nhân sử dụng hệ thống	27
3.2.2 Yêu cầu chức năng	27
3.2.3 Yêu cầu phi chức năng	28
3.3 Kiến trúc tổng thể hệ thống	29
3.4 Thiết kế thiết bị và trạng thái	29

3.4.1	Device model	29
3.4.2	State, event và command	30
3.5	Luồng dữ liệu chính	30
3.5.1	Luồng uplink: thiết bị báo cáo lên Cloud	30
3.5.2	Luồng downlink: người dùng gửi lệnh điều khiển	31
3.5.3	Luồng automation	31
3.6	Thiết kế MQTT contract	31
3.7	Thiết kế Cloud Backend và cơ sở dữ liệu	33
3.7.1	REST API	33
3.7.2	Cơ sở dữ liệu	33
3.8	Thiết kế Gateway	34
3.9	Thiết kế Mobile App	35
3.10	Các quyết định thiết kế và trade-off	35
4	TRIỂN KHAI HỆ THỐNG	36
4.1	Định hướng triển khai: từ học Zigbee đến prototype hướng product	36
4.2	Môi trường triển khai và cấu trúc mã nguồn	37
4.3	Triển khai mạng Zigbee và firmware thiết bị	38
4.3.1	Triển khai Coordinator / NCP Board	38
4.3.2	Triển khai Router đèn (Z3Light Device)	38
4.3.3	Triển khai Router công tắc (Z3Switch Device)	38
4.3.4	Triển khai cảm biến hiện diện (Motion/Occupancy Sensor)	39
4.4	Triển khai Gateway Edge	40
4.4.1	MQTT client và command queue	40
4.4.2	Command handler và điều khiển thiết bị	40
4.4.3	Discovery, registry và rule cục bộ	40
4.5	Triển khai Cloud Backend	41
4.5.1	FastAPI app lifecycle và router	41
4.5.2	Database model và command lifecycle	42
4.5.3	Cấu hình biến môi trường	42
4.5.4	MQTT service và timeout worker	43
4.6	Triển khai Mobile App	43
4.7	Triển khai môi trường chạy thực tế	45
4.8	Luồng vận hành End-to-End Command	46
4.9	Giới hạn hiện tại và hướng product hóa	47
4.10	Tổng kết chương	48
5	KIỂM THỬ VÀ ĐÁNH GIÁ	49
5.1	Kế hoạch kiểm thử	49
5.2	Môi trường và cấu hình kiểm thử	49
5.2.1	Cấu hình phần cứng thử nghiệm	49
5.3	Kết quả thực thi các kịch bản kiểm thử	51
5.3.1	Nhóm 1: Kiểm thử Gia nhập mạng của thiết bị	51
5.3.2	Nhóm 2: Kiểm thử truyền nhận trạng thái và điều khiển	51
5.3.3	Nhóm 3: Kiểm thử tự động hóa cục bộ và độ tin cậy	52
5.4	Đánh giá kết quả kiểm thử	52
5.4.1	Mức độ hoàn thành các tiêu chí hệ thống	52
5.4.2	Phân tích độ trễ của hệ thống	54
5.5	Hạn chế của hệ thống phát hiện qua kiểm thử	54

6	KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	56
6.1	Kết luận chung	56
6.2	Kiến thức và kỹ năng đạt được	56
6.3	Hướng phát triển tiếp theo	57
6.3.1	Hoàn thiện phần cứng và kiểm thử dài hạn	57
6.3.2	Nâng cấp Gateway	57
6.3.3	Nâng cấp Cloud và Mobile App	57
6.3.4	Tăng cường bảo mật	57
6.4	Kết luận chương	58
	TÀI LIỆU THAM KHẢO	59

DANH MỤC KÝ HIỆU VÀ CHỮ VIẾT TẮT

Viết tắt	Ý nghĩa
ACK	Acknowledgement
AES	Advanced Encryption Standard
API	Application Programming Interface
APS	Application Support Sub-layer
CID	Correlation ID
CLI	Command Line Interface
CMD	Command
CMSIS	Cortex Microcontroller Software Interface Standard
CRLF	Carriage Return Line Feed
CSA	Connectivity Standards Alliance
CSMA/CA	Carrier Sense Multiple Access with Collision Avoidance
DMA	Direct Memory Access
EZSP	EmberZNet Serial Protocol
FFD	Full Function Device
GBL	Gecko Bootloader
GPIO	General Purpose Input/Output
GSDK	Gecko SDK
GUI	Graphical User Interface
HTTP	Hypertext Transfer Protocol
I2C	Inter-Integrated Circuit
IDE	Integrated Development Environment
IoT	Internet of Things
JSON	JavaScript Object Notation
LCD	Liquid Crystal Display
LED	Light Emitting Diode
LQI	Link Quality Indicator
LWT	Last Will and Testament (MQTT)
MAC	Media Access Control
MCU	Microcontroller Unit
MQTT	Message Queuing Telemetry Transport
NCP	Network Co-Processor
NVM3	Non-Volatile Memory 3rd Generation
NWK	Network Layer
OTA	Over-the-Air
PAN	Personal Area Network
PHY	Physical Layer
PSA	Platform Security Architecture
PTI	Packet Trace Interface
QoS	Quality of Service
RAIL	Radio Abstraction Interface Layer
RBAC	Role-Based Access Control
REST	Representational State Transfer
RF	Radio Frequency
RFD	Reduced Function Device
RPi	Raspberry Pi

Viết tắt	Ý nghĩa
RTOS	Real-Time Operating System
SDK	Software Development Kit
SED	Sleepy End Device
SoC	System on Chip
SPI	Serial Peripheral Interface
SSv5	Simplicity Studio version 5
TCP/IP	Transmission Control Protocol / Internet Protocol
TLS	Transport Layer Security
UART	Universal Asynchronous Receiver-Transmitter
USB	Universal Serial Bus
VCOM	Virtual COM Port
WSTK	Wireless Starter Kit
ZC	Zigbee Coordinator
ZCL	Zigbee Cluster Library
ZDO	Zigbee Device Object
ZED	Zigbee End Device
ZR	Zigbee Router

DANH SÁCH HÌNH VẼ

1.1	Quy trình nghiên cứu và triển khai hệ thống	11
2.1	Kiến trúc tổng quát của hệ thống IoT Zigbee Smart Building	14
2.2	Mình họa topology dạng mesh trong mạng Zigbee	17
2.3	Kiến trúc tầng giao thức Zigbee	18
2.4	Mô hình broker trung tâm trong MQTT	20
2.5	Mình họa giao tiếp UART giữa hai thành phần phần cứng	22
3.1	Kiến trúc runtime của hệ thống Smart Building trong đồ án	29
3.2	Luồng uplink đồng bộ trạng thái và sự kiện	30
3.3	Luồng downlink và phản hồi command	31
3.4	Ranh giới thiết kế automation trong phiên bản hiện tại	31
4.1	Định hướng triển khai tích hợp giữa nghiên cứu mạng Zigbee và giải pháp sản phẩm IoT.	37
4.2	Sơ đồ cấu trúc topo mạng Zigbee mesh được xây dựng trong đồ án.	39
4.3	Kiến trúc luồng xử lý không đồng bộ bên trong phần mềm Gateway Edge.	41
4.4	Lược đồ chuyển trạng thái trong vòng đời lệnh điều khiển trên Cloud Backend.	43
4.5	Kiến trúc phân lớp cấu trúc mã nguồn bên trong ứng dụng di động Flutter.	44
4.6	Kiến trúc triển khai và luồng kết nối mạng của các thành phần trên AWS EC2.	46
4.7	Trình tự luồng dữ liệu 12 bước điều khiển thiết bị đèn từ Mobile App.	47
4.8	Biểu đồ ranh giới năng lực kỹ thuật và lộ trình sản phẩm hóa của hệ thống.	48

DANH SÁCH BẢNG

2	Phân chia công việc giữa các thành viên	8
1.1	Đóng góp chính và ý nghĩa kỹ thuật của đề án	12
2.1	Đặc điểm chính của Zigbee trong hệ thống Smart Building	15
2.2	Vai trò thiết bị trong mạng Zigbee	16
2.3	Một số cluster liên quan đến đề tài	18
2.4	Vai trò của các thành phần Cloud trong hệ thống	23
2.5	Tóm tắt công nghệ nền được sử dụng trong đề án	25
3.1	Phạm vi thiết kế trong đề án	26
3.2	Các tác nhân chính trong hệ thống	27
3.3	Các yêu cầu chức năng chính	28
3.4	Các yêu cầu phi chức năng	28
3.5	Trách nhiệm của các khối chính	29
3.6	Ánh xạ loại thiết bị và chức năng runtime	30
3.7	Các nhóm topic MQTT chính	32
3.8	Các nhóm REST API chính	33
3.9	Các bảng dữ liệu chính	34
3.10	Các module logic chính trong Gateway	34
3.11	Các trade-off chính trong thiết kế	35
4.1	Môi trường công nghệ và bản đồ mã nguồn triển khai	37
4.2	Thông tin minh chứng các tệp tin cấu hình và firmware thiết bị	39
4.3	Bản đồ phân rã chức năng các file mã nguồn Gateway C	41
4.4	Danh mục biến môi trường của dịch vụ Cloud Backend	42
4.5	Bản đồ phân rã chức năng các file mã nguồn Cloud Backend	43
4.6	Bản đồ cấu trúc thư mục mã nguồn và vai trò của Mobile App	45
4.7	Bảng so sánh hiện trạng prototype và hướng sản phẩm hóa	48
5.1	Cấu hình địa chỉ và vai trò của các thiết bị trong mạng thử nghiệm	50
5.2	Nhóm kịch bản kiểm thử gia nhập mạng (Device Join)	51
5.3	Nhóm kịch bản kiểm thử truyền tin và điều khiển (Uplink/Downlink)	52
5.4	Nhóm kịch bản kiểm thử tự động hóa và xử lý lỗi (Automation & Faults)	53
5.5	Tổng hợp đánh giá mức độ hoàn thành các tiêu chí hệ thống	54

PHÂN CHIA CÔNG VIỆC

Bảng 2 chia công việc thành hai vùng chính theo kiến trúc của hệ thống. Vùng Zigbee cục bộ bao gồm thiết bị, mạng Zigbee và Gateway. Vùng Cloud và lớp ứng dụng bao gồm dịch vụ phía máy chủ, cơ sở dữ liệu, phân quyền và ứng dụng di động.

Bảng 2: Phân chia công việc giữa các thành viên

STT	Công việc	Sinh viên thực hiện
Vùng 1: Zigbee cục bộ		
1	Nghiên cứu mạng Zigbee, vai trò Coordinator, Router và End Device; cấu hình bộ xử lý mạng EFR32 ở chế độ Network Co-Processor (NCP).	Lê Đức Anh
2	Xây dựng firmware cho thiết bị đèn: On/Off Server, phản hồi trạng thái và kiểm tra hoạt động của đèn.	Lê Đức Anh
3	Xây dựng firmware cho công tắc và cảm biến chuyển động: phát sinh sự kiện, báo cáo thuộc tính và kiểm tra tham gia mạng.	Chu Thiên Phú
4	Triển khai Gateway bằng ngôn ngữ C: kết nối Host–NCP, quản lý thiết bị, nhận dữ liệu Zigbee và điều phối lệnh.	Lê Đức Anh
5	Tích hợp Message Queuing Telemetry Transport (MQTT) trên Gateway: nhận lệnh, gửi trạng thái, sự kiện, thông tin thiết bị và phản hồi thực thi.	Lê Đức Anh, Chu Thiên Phú
Vùng 2: Cloud và lớp ứng dụng		
6	Thiết kế và triển khai Cloud Backend bằng FastAPI; xây dựng các Application Programming Interface (API) cho thiết bị, lệnh, sự kiện, tự động hóa và thêm thiết bị.	Chu Thiên Phú
7	Thiết kế lược đồ cơ sở dữ liệu cho người dùng, nhà, phòng, thiết bị, trạng thái, sự kiện, lệnh và luật tự động hóa.	Chu Thiên Phú
8	Triển khai đăng nhập, quản lý phiên và Role-Based Access Control (RBAC), tức cơ chế phân quyền theo vai trò và phạm vi nhà/thiết bị.	Chu Thiên Phú
9	Phát triển Mobile App bằng Flutter: đăng nhập, giám sát thiết bị, điều khiển đèn, xem sự kiện, tự động hóa và thêm thiết bị.	Lê Đức Anh
10	Tích hợp xuyên suốt Device–Gateway–Cloud–App; xây dựng test case, thu thập evidence và xử lý lỗi liên thành phần.	Lê Đức Anh, Chu Thiên Phú
11	Quản lý công việc bằng Jira; quản lý phiên bản mã nguồn bằng Git và GitHub; tổng hợp tài liệu và hoàn thiện báo cáo.	Lê Đức Anh, Chu Thiên Phú

Việc phân chia trên xác định người phụ trách chính nhưng không tách rời quá trình tích hợp. Các luồng xuyên suốt, đặc biệt là điều khiển đèn, đồng bộ trạng thái và tự động hóa, cần cả hai thành viên cùng kiểm tra vì dữ liệu đi qua nhiều lớp của hệ thống.

CHƯƠNG 1

TỔNG QUAN ĐỀ TÀI

Chương này trình bày bối cảnh và lý do lựa chọn đề tài xây dựng hệ thống IoT Zigbee Smart Building. Tiếp theo, chương xác định rõ mục tiêu tổng quát, mục tiêu cụ thể, phạm vi nghiên cứu và các giới hạn thực tế của đề án. Cuối cùng, phương pháp thực hiện, những đóng góp chính về mặt kỹ thuật và cấu trúc tổng thể của báo cáo cũng được khái quát hóa nhằm cung cấp cái nhìn toàn cảnh trước khi đi vào các nội dung kỹ thuật chi tiết.

1.1 Bối cảnh và lý do chọn đề tài

Trong những năm gần đây, sự phát triển mạnh mẽ của công nghệ mạng không dây và Internet vạn vật (Internet of Things - IoT) đã thúc đẩy việc ứng dụng tự động hóa vào quản lý vận hành tòa nhà. Smart Building (tòa nhà thông minh) không còn là khái niệm lý thuyết mà đã trở thành xu hướng tất yếu nhằm tối ưu hóa lượng điện năng tiêu thụ, nâng cao độ an toàn và mang lại trải nghiệm tiện nghi cho người sử dụng. Trong các hệ thống này, nhu cầu giám sát trạng thái môi trường, phát hiện sự hiện diện của con người và điều khiển các thiết bị phụ tải (như đèn chiếu sáng, công tắc) là những bài toán cốt lõi cần giải quyết.

Đối với mạng kết nối nội bộ (local network) trong tòa nhà, việc sử dụng các công nghệ truyền thông không dây phổ biến như Wi-Fi thường bộc lộ những hạn chế lớn về mặt tiêu thụ năng lượng và khả năng chịu tải của điểm truy cập trung tâm khi số lượng thiết bị lên tới hàng trăm node. Do đó, các công thức truyền thông tiết kiệm năng lượng, hỗ trợ cấu trúc liên kết mạng lưới (mesh network) như Zigbee được xem là giải pháp tối ưu. Zigbee hoạt động trên tiêu chuẩn IEEE 802.15.4, cho phép các thiết bị tự động định tuyến gói tin qua nhau, giúp mở rộng phạm vi phủ sóng mà không yêu cầu công suất phát lớn, đồng thời giảm thiểu đáng kể chi phí duy trì nguồn pin cho các cảm biến.

Tuy nhiên, các mạng Zigbee cục bộ không thể tự kết nối trực tiếp với các ứng dụng di động hoặc hệ thống quản lý từ xa qua mạng IP diện rộng. Điều này đòi hỏi sự xuất hiện của một thiết bị trung gian gọi là Gateway. Thiết bị Gateway đóng vai trò làm cầu nối chuyển đổi giữa giao thức Zigbee RF tầm ngắn và mạng IP (chạy các giao thức như MQTT, HTTP) kết nối lên Cloud. Sự phối hợp giữa Gateway và hệ thống đám mây (Cloud Backend) cho phép lưu trữ lịch sử vận hành, xử lý các kịch bản tự động hóa phức tạp và cung cấp các giao diện lập trình ứng dụng (API) cho ứng dụng di động của người dùng cuối.

Hơn nữa, các giải pháp smart building thương mại hiện nay thường là các hệ sinh thái đóng (closed ecosystem) của các nhà sản xuất lớn, gây khó khăn cho việc tích hợp thiết bị của bên thứ ba và hạn chế khả năng tùy biến logic điều khiển. Xuất phát từ nhu cầu nghiên cứu một nền tảng quản lý thiết bị mở, có khả năng làm chủ từ firmware thiết bị, logic xử lý tại gateway cho đến kiến trúc cloud backend, đề tài "**Hệ thống IoT Zigbee Smart Building**" đã được lựa chọn để thực hiện. Đề án tập trung nghiên cứu thiết kế và tích hợp đồng bộ các lớp thiết bị, gateway và dịch vụ đám mây nhằm đảm bảo luồng dữ liệu hai chiều nhanh chóng, tin cậy và có khả năng mở rộng.

1.2 Mục tiêu của đề án

Để giải quyết các bài toán đặt ra trong bối cảnh trên, đề án xác định rõ mục tiêu tổng quát và các mục tiêu cụ thể cần đạt được.

1.2.1 Mục tiêu tổng quát

Mục tiêu tổng quát của đề án là thiết kế, chế tạo và tích hợp hoàn chỉnh một hệ thống IoT quản lý Smart Building dựa trên chuẩn truyền thông không dây Zigbee, kết nối qua Gateway C tới hệ thống Cloud Backend, hỗ trợ giám sát trạng thái thiết bị thực tế gần thời gian thực và tự động hóa điều khiển phụ tải.

1.2.2 Mục tiêu cụ thể

Để hiện thực hóa mục tiêu tổng quát, đề án đề ra các mục tiêu cụ thể cho từng thành phần trong hệ thống:

Phía thiết bị cuối (End Devices): Thiết kế và nạp firmware cho các thiết bị Zigbee 3.0 dựa trên vi điều khiển EFR32MG12 bao gồm thiết bị đèn (Z3Light), công tắc (Z3Switch) và cảm biến hiện diện (Occupancy/Motion Sensor) hoạt động ổn định trong mạng mesh Zigbee.

Phía Gateway trung gian: Triển khai ứng dụng Z3Gateway dạng tiến trình đơn (single-process) viết bằng ngôn ngữ C chạy trên Linux host. Tích hợp trực tiếp thư viện kết nối MQTT để giao tiếp trực tiếp với Cloud, loại bỏ hoàn toàn các lớp trung gian trung chuyển IPC phức tạp nhằm giảm độ trễ và tăng tính ổn định.

Phía dịch vụ đám mây (Cloud Backend): Xây dựng Cloud Backend sử dụng framework FastAPI (Python 3.12) kết nối cơ sở dữ liệu PostgreSQL. Hệ thống đám mây phải tiếp nhận các bản tin telemetry báo cáo trạng thái, quản lý vòng đời lệnh điều khiển, lưu trữ lịch sử sự kiện và cung cấp hệ thống REST API chuẩn hóa.

Phía ứng dụng giám sát (Dashboard/App): Thiết kế các giao diện mô phỏng dưới dạng API hoặc màn hình quản lý cho phép theo dõi trực quan danh sách thiết bị, trạng thái vận hành hiện tại và gửi lệnh bật/tắt thiết bị đèn.

Về mặt kiểm thử và đánh giá: Xây dựng bộ kịch bản kiểm thử (test cases) end-to-end hoàn chỉnh nhằm đo đặc độ trễ phản hồi của lệnh điều khiển, kiểm chứng tính chính xác của cơ chế đồng bộ trạng thái và ghi nhận bằng chứng vận hành (logs, traces).

1.3 Phạm vi và giới hạn

Việc xác định phạm vi nghiên cứu và giới hạn thực thi giúp đồ án tập trung vào các vấn đề kỹ thuật trọng tâm, tránh phân tán nguồn lực vào các tính năng thương mại phức tạp.

1.3.1 Phạm vi nghiên cứu và triển khai

Đồ án tập trung thực hiện các nội dung sau:

Thiết lập mạng Zigbee cục bộ: Sử dụng một Coordinator chạy firmware NCP (Network Co-Processor) kết nối qua UART/ASH với Gateway host.

Triển khai kịch bản gia nhập mạng: Thực hiện permit-join, đồng bộ trạng thái thực tế lên cloud (reported state) và truyền đạt lệnh trạng thái mong muốn (desired state) qua giao thức MQTT.

Đồng bộ trạng thái thiết bị và kịch bản tự động hóa cục bộ: Hỗ trợ local automation cơ bản thông qua cơ chế APS Binding trực tiếp giữa công tắc và đèn trong mạng Zigbee mesh.

Theo dõi vòng đời của lệnh điều khiển: Theo dõi chặt chẽ quá trình xử lý lệnh từ Cloud xuống thiết bị qua các pha trạng thái rõ ràng (accepted → queued → sent → executed/failed/-timeout).

Triển khai hệ thống Cloud Backend: Deploy hệ thống trên hạ tầng điện toán đám mây dưới dạng các container Docker được đóng gói và cấu hình hoàn chỉnh.

1.3.2 Giới hạn của đề tài

Do hạn chế về mặt thời gian và trang thiết bị phần cứng, đồ án tồn tại một số giới hạn như sau:

Quy mô thử nghiệm: Do hạn chế về mặt thời gian và trang thiết bị phần cứng, hệ thống chỉ dừng lại ở quy mô thử nghiệm phòng lab (lab-scale) với số lượng thiết bị demo hạn chế (khoảng 3-5 node), chưa được triển khai thực tế trên quy mô tòa nhà nhiều tầng với mật độ cản trở vật lý cao.

Ứng dụng di động: Ứng dụng di động Flutter và cơ chế điều khiển tích hợp qua giao thức Bluetooth Low Energy (BLE) chỉ dừng lại ở mức thiết kế giao diện tĩnh (placeholders) hoặc mô phỏng logic client, chưa được tích hợp kết nối API thực tế chạy trên thiết bị di động thật trong bản v1 này.

Tính năng nâng cao: Các tính năng nâng cao như cơ chế cập nhật firmware từ xa (OTA Update), bảo mật mã hóa đường truyền MQTT qua TLS (port 8883) và hệ thống xác thực người dùng (Authentication API) chỉ được thực hiện ở mức phân tích thiết kế kịch bản lý thuyết (contracts), chưa có mã nguồn triển khai chạy thực tế.

1.4 Phương pháp thực hiện

Để thực hiện đồ án một cách khoa học, quy trình nghiên cứu và phát triển được chia làm 5 bước tuần tự như được mô tả trong Hình 1.1.

Hình 1.1: Quy trình nghiên cứu và triển khai hệ thống

Quy trình thực hiện bao gồm các giai đoạn cụ thể sau:

Giai đoạn 1 – Khảo sát và nghiên cứu công nghệ: Nghiên cứu tài liệu kỹ thuật về chuẩn Zigbee 3.0, thư viện ZCL (Zigbee Cluster Library) của Silicon Labs, giao thức truyền thông

MQTT và kiến trúc API hướng tài nguyên sử dụng REST.

Giai đoạn 2 – Thiết kế hệ thống: Xác định sơ đồ khối kiến trúc hệ thống 4 lớp, thiết kế cấu trúc cơ sở dữ liệu PostgreSQL, xây dựng hợp đồng trao đổi bản tin MQTT (MQTT Contract JSON Schema) và cấu hình các topic phân cấp.

Giai đoạn 3 – Triển khai chi tiết từng module: Lập trình firmware cho các SoC EFR32MG12; viết mã nguồn Z3Gateway bằng ngôn ngữ C; xây dựng Cloud API bằng FastAPI.

Giai đoạn 4 – Tích hợp hệ thống: Kết nối Gateway host với chip NCP qua UART, kết nối Gateway lên Mosquitto Broker chạy trên container Docker, thiết lập liên kết đồng bộ giữa cơ sở dữ liệu và MQTT Client chạy nền trên Cloud.

Giai đoạn 5 – Kiểm thử và đánh giá: Chạy các kịch bản kiểm thử tích hợp tự động bằng pytest cho backend và thực thi kiểm thử thủ công end-to-end trực tiếp trên phần cứng thật để thu thập log vận hành.

1.5 Đóng góp chính của đề án

Kết quả thực hiện đề án mang lại một số đóng góp chính về mặt giải pháp kỹ thuật tự phát triển thay vì sử dụng các framework đóng gói sẵn của hãng. Bảng 1.1 tóm tắt các đóng góp và ý nghĩa thực tế tương ứng của chúng.

Bảng 1.1: Đóng góp chính và ý nghĩa kỹ thuật của đề án

Giải pháp phát triển	Ý nghĩa thực tế và kỹ thuật
Kiến trúc Gateway C đơn tiến trình tích hợp trực tiếp MQTT client	Loại bỏ hoàn toàn tiến trình trung gian Python IPC cũ, giúp giảm thiểu độ trễ truyền dẫn lệnh xuống thiết bị và tiết kiệm tài nguyên CPU/RAM trên thiết bị host nhúng.
Mô hình theo dõi vòng đời lệnh điều khiển (Command Lifecycle)	Cho phép theo dõi tường minh trạng thái xử lý gói tin qua các pha (accepted, queued, sent, executed, failed, timeout), giải quyết bài toán lệnh bị treo hoặc mất gói vô tuyến mà cloud không phát hiện được.
Chuẩn hóa mô hình nhận dạng thiết bị qua device_id logic	Tách biệt định danh cơ sở dữ liệu khỏi địa chỉ phần cứng cố định (EUI-64) hoặc địa chỉ động (16-bit nwk_addr), tăng tính bảo mật và cho phép linh hoạt thay thế phần cứng lỗi.
Bộ kịch bản kiểm thử E2E tích hợp evidence thực tế	Cung cấp phương pháp luận kiểm thử rõ ràng từ mức đơn vị đến tích hợp hệ thống, đi kèm các bằng chứng vận hành (log trace) chứng minh hệ thống hoạt động chính xác.

Như thể hiện trong Bảng 1.1, mỗi đóng góp kỹ thuật trong đề án đều giải quyết một hạn chế cụ thể của kiến trúc cũ hoặc mang lại giá trị ứng dụng rõ ràng. Trong đó, kiến trúc Gateway C đơn tiến trình và mô hình theo dõi vòng đời lệnh là hai đóng góp cốt lõi quyết định tính ổn định của toàn hệ thống. Các đóng góp trên được minh chứng thông qua kết quả kiểm thử thực tế tại Chương 5, cho thấy tính khả thi và độ tin cậy của giải pháp tự phát triển trong môi trường nghiên cứu phòng thí nghiệm.

1.6 Cấu trúc báo cáo

Nội dung báo cáo đồ án tốt nghiệp được tổ chức thành 6 chương chính sau đây:

Chương 1 – Tổng quan đề tài: Giới thiệu bối cảnh, lý do chọn đề tài, xác định mục tiêu, phạm vi nghiên cứu, phương pháp thực hiện và cấu trúc tổng thể của báo cáo.

Chương 2 – Cơ sở lý thuyết và công nghệ nền: Trình bày nền tảng lý thuyết về giao thức mạng Zigbee, thư viện cluster ZCL, giao thức MQTT, giao tiếp nối tiếp UART/ASH và các kit vi điều khiển phần cứng được sử dụng.

Chương 3 – Phân tích yêu cầu và thiết kế hệ thống: Phân tích các yêu cầu chức năng, phi chức năng, xây dựng sơ đồ kiến trúc tổng thể, mô hình dữ liệu, đặc tả giao diện API và hợp đồng trao đổi bản tin MQTT Contract.

Chương 4 – Triển khai hệ thống: Trình bày chi tiết quá trình lập trình firmware thiết bị, viết ứng dụng Gateway C, lập trình Cloud Backend bằng FastAPI và giải quyết các lỗi thực tế khi deploy lên EC2.

Chương 5 – Kiểm thử và đánh giá: Thiết lập kế hoạch kiểm thử, trình bày kết quả chạy các test case cụ thể, ghi nhận các bằng chứng logs/traces và phân tích ưu điểm cũng như hạn chế của hệ thống.

Chương 6 – Kết luận và hướng phát triển: Tổng kết các kết quả đạt được, tự đánh giá năng lực tích lũy của bản thân và đề xuất các hướng nghiên cứu mở rộng trong tương lai như bảo mật TLS, cập nhật OTA và tích hợp WebSocket.

CHƯƠNG 2

CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ NỀN

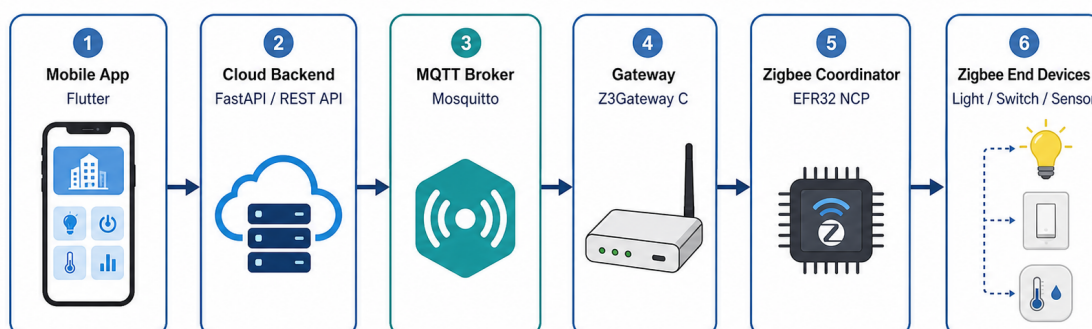
Chương 2 trình bày các cơ sở lý thuyết và công nghệ được sử dụng để xây dựng hệ thống IoT Zigbee Smart Building. Nội dung gồm mô hình IoT trong tòa nhà thông minh, mạng Zigbee, mô hình thiết bị theo Zigbee Cluster Library, vai trò của Gateway, giao thức MQTT, giao tiếp giữa Host và EFR32 NCP, Cloud Backend, Mobile App, firmware/OTA và một số vấn đề bảo mật cơ bản. Các phần dưới đây được trình bày ở mức nền tảng để làm cơ sở cho phần thiết kế và triển khai ở các chương sau, không đi vào log chạy thử hay kết quả thực nghiệm.

2.1 Tổng quan IoT và Smart Building

Internet of Things (IoT), hay Internet vạn vật, là mô hình kết nối các thiết bị vật lý với phần mềm thông qua mạng truyền thông. Thiết bị trong hệ thống IoT thường có cảm biến, bộ xử lý, phần mềm nhúng và khả năng trao đổi dữ liệu với các thành phần khác. Trong phạm vi đồ án, IoT được xét theo hướng ứng dụng trong một phòng/lab thông minh, nơi thiết bị có thể gửi trạng thái, nhận lệnh điều khiển và phối hợp với phần mềm phía trên.

Smart Building là một hướng ứng dụng của IoT trong môi trường tòa nhà. Hệ thống thường bao gồm sensor (cảm biến), actuator (thiết bị chấp hành), Gateway (cổng kết nối), Cloud Backend (dịch vụ phía máy chủ) và App (ứng dụng người dùng). Sensor dùng để thu thập trạng thái, ví dụ cảm biến hiện diện phát hiện có người trong phòng. Actuator thực hiện hành động vật lý như bật/tắt đèn hoặc thay đổi trạng thái thiết bị. Gateway nối mạng thiết bị cục bộ với hệ thống phần mềm. Cloud Backend lưu dữ liệu, quản lý thiết bị và cung cấp Application Programming Interface (API) cho App.

Một luồng IoT cơ bản thường bắt đầu từ việc thu thập dữ liệu, truyền dữ liệu, xử lý dữ liệu, phát sinh lệnh điều khiển và hiển thị lại trạng thái cho người dùng. Trong đồ án này, người dùng thao tác trên Flutter Mobile App; App gọi FastAPI Cloud REST API; Cloud trao đổi trạng thái và lệnh qua Mosquitto MQTT Broker; Z3Gateway C nhận bản tin MQTT và chuyển thành Zigbee command qua EFR32 NCP; thiết bị Zigbee phản hồi trạng thái theo chiều ngược lại.



Hình 2.1: Kiến trúc tổng quát của hệ thống IoT Zigbee Smart Building

Cách tổ chức trên giúp tách rõ trách nhiệm giữa các lớp. Mạng Zigbee xử lý truyền thông cục bộ cho thiết bị công suất thấp. Gateway đảm nhiệm chuyển đổi giữa Zigbee và MQTT. Cloud lưu dữ liệu và cung cấp API. App tập trung vào giao diện giám sát và điều khiển. Khi thiết kế hệ thống Smart Building, nhóm thực hiện cân cân bằng giữa xử lý cục bộ và xử lý trên Cloud. Xử lý cục bộ giúp giảm độ trễ trong một số tình huống, còn Cloud thuận lợi hơn cho lưu trữ, quản lý từ xa và mở rộng dữ liệu.

2.2 Zigbee và IEEE 802.15.4

Zigbee là giao thức truyền thông không dây công suất thấp, thường được dùng trong các hệ thống cảm biến và điều khiển [1]. Đặc điểm của Zigbee là tốc độ dữ liệu không cao nhưng tiêu thụ năng lượng thấp, hỗ trợ nhiều thiết bị và có khả năng tổ chức mạng linh hoạt. Các đặc điểm này phù hợp với Smart Building, vì các thiết bị như đèn, công tắc và cảm biến hiện diện chủ yếu trao đổi trạng thái hoặc lệnh ngắn, không truyền dữ liệu dung lượng lớn.

IEEE 802.15.4 là chuẩn ở tầng vật lý và tầng Medium Access Control (MAC) cho mạng cá nhân không dây công suất thấp. Zigbee sử dụng nền tảng này và bổ sung thêm các tầng phía trên như network layer, security và application layer [3]. Có thể hiểu IEEE 802.15.4 quy định cách truyền nhận khung dữ liệu ở mức radio, còn Zigbee quy định thêm cách thiết bị tạo mạng, tham gia mạng, định tuyến và biểu diễn chức năng ứng dụng.

Trong một hệ thống IoT, Zigbee không thay thế hoàn toàn Wi-Fi hoặc Ethernet. Zigbee phù hợp hơn ở lớp thiết bị cục bộ, nơi yêu cầu tiết kiệm năng lượng và truyền các bản tin nhỏ. Đối lại, thiết bị Zigbee thường không giao tiếp trực tiếp với Internet, nên hệ thống cần Gateway làm cầu nối lên Cloud hoặc App.

Bảng 2.1: Đặc điểm chính của Zigbee trong hệ thống Smart Building

Đặc điểm	Ý nghĩa trong hệ thống
Công suất thấp	Phù hợp với cảm biến, công tắc và thiết bị nhúng cần hoạt động lâu dài.
Băng thông thấp	Không phù hợp truyền dữ liệu lớn, nhưng đủ cho trạng thái, sự kiện và lệnh điều khiển.
Hỗ trợ mesh	Có thể mở rộng vùng phủ thông qua các thiết bị có khả năng định tuyến.
Mô hình ZCL	Chuẩn hóa chức năng thiết bị thành cluster, attribute và command.
Cần Gateway	Thiết bị Zigbee không giao tiếp trực tiếp với App/Cloud qua Internet, nên cần Gateway làm cầu nối.

Bảng 2.1 tóm tắt các đặc điểm nổi bật của Zigbee và mối liên hệ trực tiếp của chúng với yêu cầu vận hành trong hệ thống Smart Building. Đặc biệt, việc Zigbee yêu cầu Gateway làm cầu nối lên mạng Internet là điểm khởi đầu quan trọng dẫn đến quyết định thiết kế môi trường triển khai theo mô hình đa lớp của đề án. EFR32 NCP đóng vai trò radio coordinator, còn các thiết bị như Light, Switch và Occupancy Sensor là các thiết bị Zigbee trong mạng. Nhờ đó, phần thiết bị nhúng được tách khỏi phần Cloud/App, đồng thời vẫn có thể đồng bộ trạng thái và nhận lệnh điều khiển thông qua Gateway.

2.3 Vai trò thiết bị trong mạng Zigbee

Mạng Zigbee phân chia thiết bị theo vai trò để mạng có thể hình thành, duy trì kết nối và truyền dữ liệu. Ba vai trò cơ bản gồm Coordinator, Router và End Device. Coordinator tạo và quản lý mạng. Router có thể chuyển tiếp gói tin để mở rộng vùng phủ. End Device là thiết bị cuối, thực hiện chức năng ứng dụng như đo trạng thái hoặc nhận lệnh điều khiển.

Coordinator là thiết bị khởi tạo mạng Zigbee. Thiết bị này chọn kênh, thiết lập Personal Area Network (PAN), quản lý quá trình cho phép thiết bị khác tham gia mạng và thường liên quan đến Trust Center trong cơ chế bảo mật. Trong kiến trúc Host-NCP, vai trò coordinator radio có thể đặt trên EFR32 NCP, còn ứng dụng Gateway chạy ở phía Host.

Router là thiết bị có khả năng chuyển tiếp gói tin cho các node khác. Trong môi trường tòa nhà, Router giúp mở rộng vùng phủ khi thiết bị không thể liên lạc trực tiếp với Coordinator. Tuy nhiên, Router thường cần nguồn cấp ổn định hơn so với End Device vì phải duy trì chức năng định tuyến.

End Device là thiết bị cuối trong mạng. Một End Device có thể là cảm biến hiện diện, công tắc hoặc đèn. Thiết bị này tập trung vào chức năng ứng dụng, ví dụ gửi trạng thái có người/không có người hoặc nhận lệnh bật/tắt. Với các thiết bị dùng pin, End Device có thể được tối ưu để tiết kiệm năng lượng bằng cách ngủ trong một số khoảng thời gian.

Bảng 2.2: Vai trò thiết bị trong mạng Zigbee

Vai trò	Nhiệm vụ chính	Ví dụ trong đề tài
Coordinator	Tạo mạng, quản lý mạng và hỗ trợ thiết bị tham gia mạng.	EFR32 NCP gắn với Gateway.
Router	Chuyển tiếp gói tin, mở rộng vùng phủ mạng.	Thiết bị luôn cấp nguồn nếu được cấu hình hỗ trợ routing.
End Device	Thực hiện chức năng ứng dụng, gửi trạng thái hoặc nhận lệnh.	Light, Switch, Occupancy Sensor.

Phần này không đi sâu vào thuật toán định tuyến của Zigbee. Mục tiêu là làm rõ vì sao Gateway cần một coordinator radio để quản lý mạng, và vì sao các thiết bị như light, switch, sensor có thể được tổ chức như các node trong cùng một mạng Zigbee.

2.4 Topology mạng Zigbee

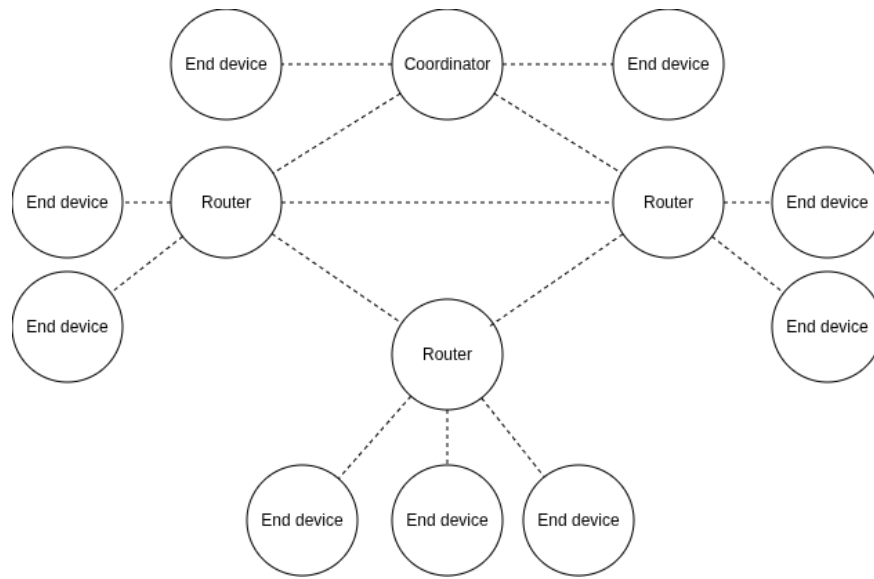
Topology mô tả cách các node trong mạng được tổ chức và liên kết với nhau. Trong Zigbee, ba dạng topology thường gặp là star, tree và mesh. Việc lựa chọn topology ảnh hưởng trực tiếp đến vùng phủ, độ phức tạp khi quản lý mạng và khả năng mở rộng của hệ thống.

Với star topology, các thiết bị trao đổi dữ liệu thông qua một node trung tâm. Mô hình này đơn giản, dễ kiểm soát và phù hợp với phạm vi nhỏ. Hạn chế của nó là vùng phủ phụ thuộc nhiều vào node trung tâm; nếu thiết bị ở xa hoặc bị vật cản, chất lượng kết nối có thể giảm.

Tree topology tổ chức mạng theo dạng phân cấp. Cách tổ chức này phù hợp khi muốn quản lý theo nhánh, ví dụ theo tầng hoặc theo khu vực trong tòa nhà. Hạn chế là các node phía dưới phụ thuộc vào node trung gian trong nhánh; khi node trung gian gặp sự cố, các node phía sau có thể bị ảnh hưởng.

Mesh topology cho phép một số node chuyển tiếp gói tin qua nhiều đường khác nhau. Đây là topology đáng chú ý trong Smart Building vì môi trường tòa nhà thường có tường, vật cản và

khoảng cách giữa các thiết bị. Mesh giúp mở rộng vùng phủ tốt hơn star topology, nhưng việc quản lý mạng và debug cũng phức tạp hơn.



Hình 2.2: Minh họa topology dạng mesh trong mạng Zigbee

Trong đồ án, Zigbee được lựa chọn để tạo mạng cục bộ cho các thiết bị trong phòng/lab. Dù số lượng thiết bị ở giai đoạn đồ án chưa lớn, nền tảng mesh của Zigbee vẫn có ý nghĩa vì nó cho phép hệ thống mở rộng sang nhiều node hơn khi cần.

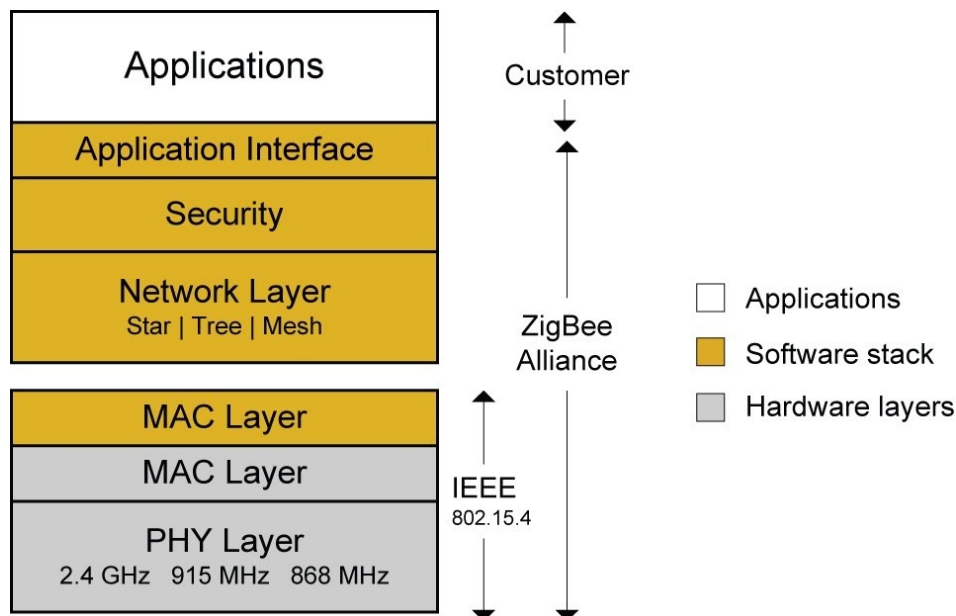
2.5 Zigbee Protocol Stack

Protocol Stack là tập hợp các tầng giao thức cùng tham gia vào quá trình truyền thông. Mỗi tầng đảm nhiệm một nhóm chức năng riêng, từ truyền tín hiệu vô tuyến ở tầng thấp đến biểu diễn lệnh ứng dụng ở tầng cao. Cách phân tầng này giúp hệ thống mạng dễ chuẩn hóa hơn, đồng thời giúp người phát triển xác định lỗi đang nằm ở tầng truyền dẫn, tầng mạng hay tầng ứng dụng.

Tầng Physical (PHY) và MAC của Zigbee dựa trên IEEE 802.15.4. Tầng PHY liên quan đến tần số, kênh truyền và việc phát/thu tín hiệu vô tuyến. Tầng MAC quản lý cách các node truy cập môi trường truyền, định dạng khung dữ liệu cơ bản và một số cơ chế trao đổi ở mức liên kết.

Tầng Network xử lý địa chỉ mạng, quá trình hình thành mạng, quá trình tham gia mạng và định tuyến. Khi một thiết bị mới join mạng, tầng này hỗ trợ cấp địa chỉ và xác định cách thiết bị gửi dữ liệu đến node đích. Với topology mesh, tầng Network có vai trò đáng kể vì gói tin có thể đi qua node trung gian.

Ở phía trên, Application Support (APS) và Application/ZCL xử lý truyền thông ở mức ứng dụng. APS hỗ trợ addressing và binding, còn ZCL định nghĩa cluster, attribute và command. Nhờ đó, lệnh bật/tắt đèn trong hệ thống Zigbee được biểu diễn theo mô hình chuẩn, thay vì dùng chuỗi dữ liệu tự do do từng nhóm tự đặt.



Hình 2.3: Kiến trúc tầng giao thức Zigbee

Ưu điểm của Protocol Stack chuẩn là tính tương thích và khả năng mở rộng tốt hơn. Hạn chế là người phát triển phải nắm thêm nhiều tầng giao thức khi debug. Với đồ án này, lợi ích của việc dùng stack chuẩn lớn hơn độ phức tạp phát sinh, vì hệ thống cần làm việc với nhiều loại thiết bị và cần ánh xạ rõ chức năng của từng thiết bị.

2.6 Zigbee Cluster Library và mô hình thiết bị

Zigbee Cluster Library (ZCL) là bộ đặc tả chuẩn hóa cách mô tả chức năng của thiết bị Zigbee [2]. Nếu các tầng thấp của Zigbee quy định cách dữ liệu đi qua mạng, ZCL quy định dữ liệu đó mang ý nghĩa gì ở tầng ứng dụng. Đây là phần cần thiết trong đồ án vì hệ thống sử dụng các thiết bị như Light, Switch và Occupancy Sensor.

Trong ZCL, cluster là nhóm chức năng của thiết bị. Attribute là trạng thái hoặc thuộc tính nằm trong cluster. Command là lệnh được gửi đến thiết bị hoặc sự kiện do thiết bị phát ra. Ba khái niệm này tạo ra cách biểu diễn thống nhất giữa Gateway và thiết bị Zigbee. Ví dụ, lệnh bật/tắt đèn có thể được biểu diễn bằng On/Off Cluster thay vì một chuỗi text tự định nghĩa.

Bảng 2.3: Một số cluster liên quan đến đèn

Cluster	Identifier	Vai trò
On/Off Cluster	0x0006	Biểu diễn chức năng bật/tắt, phù hợp với light hoặc switch.
Level Control Cluster	0x0008	Biểu diễn mức độ, ví dụ độ sáng của đèn nếu thiết bị hỗ trợ.
Occupancy Sensing Cluster	0x0406	Biểu diễn trạng thái phát hiện hiện diện, phù hợp với occupancy sensor.

Bảng 2.3 liệt kê ba cluster ZCL chính sử dụng trong đồ án. Mỗi cluster được xác định bằng một identifier (ID) cố định theo tiêu chuẩn Zigbee, đảm bảo rằng Gateway có thể phân biệt và xử lý chính xác từng loại thiết bị thông qua các bản tin báo cáo trạng thái hoặc lệnh điều khiển

tương ứng. Trong luồng điều khiển, App tạo yêu cầu ở mức người dùng, Cloud chuyển yêu cầu thành command logic, Gateway ánh xạ command sang action Zigbee, Zigbee stack đóng gói action theo cluster/command và thiết bị thực hiện lệnh. Ở chiều ngược lại, thiết bị gửi trạng thái mới, Gateway nhận báo cáo và publish reported state để Cloud lưu trữ và App hiển thị.

Việc sử dụng ZCL khiến phần phát triển phức tạp hơn so với tự đặt một payload đơn giản. Tuy nhiên, ZCL giúp hệ thống tránh tình trạng mỗi loại thiết bị dùng một cách biểu diễn riêng. Với đồ án này, lợi ích đó thể hiện rõ ở việc Gateway có thể xử lý lệnh theo loại thiết bị và chức năng, ví dụ tách lệnh bật/tắt light khỏi trạng thái phát hiện hiện diện của sensor.

2.7 Gateway trong hệ thống IoT

Gateway là thành phần nối mạng thiết bị cục bộ với hệ thống phần mềm phía trên. Trong hệ thống IoT Zigbee, thiết bị Zigbee không trực tiếp giao tiếp với REST API hoặc App. Gateway vì vậy cần chuyển đổi giữa hai miền: một bên là mạng Zigbee với cluster, attribute và command; bên còn lại là Cloud, MQTT và các dữ liệu ứng dụng.

Trong đồ án này, Gateway được hiểu theo kiến trúc hiện tại: Z3Gateway C Host App kết nối với EFR32 NCP và tích hợp MQTT trực tiếp thông qua libmosquitto. Kiến trúc Python MQTT-to-IPC bridge cũ không được dùng làm mô hình chính cho báo cáo. Luồng đúng của hệ thống là Cloud/App trao đổi qua MQTT Broker; Z3Gateway C nhận command từ MQTT; Gateway gọi Zigbee action qua Ember AF/EZSP; thiết bị Zigbee phản hồi trạng thái; Gateway publish reported state hoặc event trở lại MQTT.

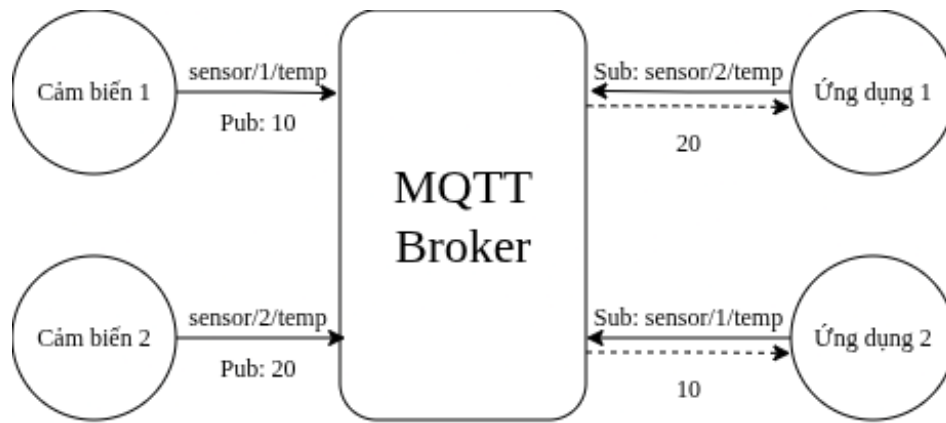
Gateway không chỉ chuyển tiếp dữ liệu thô. Thành phần này còn parse payload, xác định loại thiết bị, ánh xạ command theo device_type, gọi API của Zigbee stack, chuẩn hóa telemetry và publish trạng thái. Trong một số trường hợp, Gateway cũng có thể xử lý local automation ở mức cơ bản, ví dụ sự kiện từ switch hoặc sensor dẫn đến lệnh điều khiển light.

Khi đặt nhiều logic ở Gateway, hệ thống có thể phản hồi nhanh hơn và giảm phụ thuộc vào Cloud trong một số tình huống. Tuy nhiên, Gateway cũng trở thành thành phần khó bảo trì hơn nếu trách nhiệm bị mở rộng quá nhiều. Vì vậy, ranh giới hợp lý là Gateway xử lý kết nối thiết bị và các phản ứng cục bộ cần thiết, còn Cloud đảm nhiệm lưu trữ, API, lịch sử và quản lý người dùng.

2.8 MQTT và cơ chế Publish/Subscribe

Message Queuing Telemetry Transport (MQTT) là giao thức messaging nhẹ, được sử dụng phổ biến trong IoT để truyền telemetry và command giữa các thành phần phân tán [9, 10]. MQTT hoạt động theo mô hình publish/subscribe. Client không gửi dữ liệu trực tiếp cho nhau, mà publish message vào topic; broker nhận message và chuyển tiếp cho các client đang subscribe topic đó.

Các thuật ngữ cơ bản của MQTT gồm broker, client, topic, payload, publish, subscribe, Quality of Service (QoS), retained message và Last Will and Testament (LWT). Broker là trung tâm điều phối message. Client là thành phần kết nối tới broker, chẳng hạn Cloud Backend hoặc Gateway. Topic là đường dẫn logic dùng để phân loại message. Payload là nội dung message. Publish là hành động gửi message, còn subscribe là đăng ký nhận message.



Hình 2.4: Mô hình broker trung tâm trong MQTT

Trong hệ thống của đồ án, Cloud và Gateway cùng kết nối tới Mosquitto MQTT Broker. Ở chiều điều khiển, App gọi REST API lên Cloud, Cloud publish command request xuống topic mà Gateway subscribe. Gateway nhận message, parse payload và chuyển thành Zigbee action. Ở chiều trạng thái, Gateway nhận reported state hoặc event từ mạng Zigbee, sau đó publish lên MQTT để Cloud lưu vào database và App hiển thị lại cho người dùng.

QoS quy định mức đảm bảo truyền message. QoS 0 gửi tối đa một lần, nhẹ nhất nhưng có thể mất message. QoS 1 đảm bảo message đến ít nhất một lần, nhưng bên nhận cần xử lý khả năng trùng lặp. QoS 2 đảm bảo đúng một lần ở mức giao thức, nhưng phức tạp và tốn tài nguyên hơn. Retained message cho phép broker giữ message cuối cùng của một topic, nhờ đó subscriber mới có thể nhận trạng thái gần nhất. LWT giúp broker phát message thay mặt client nếu client mất kết nối bất thường.

MQTT phù hợp với đồ án vì bản tin trạng thái và lệnh điều khiển thường nhỏ, gửi theo sự kiện và cần tách rời Cloud với Gateway. Hạn chế chính là hệ thống phải thiết kế topic và payload rõ ràng. Nếu contract MQTT không ổn định, việc debug và mở rộng hệ thống sẽ khó khăn hơn. Vì vậy, MQTT được xem là contract chính giữa Cloud và Gateway, còn chi tiết topic/payload cụ thể được trình bày ở phần thiết kế và triển khai.

2.9 UART, ASH và EZSP giữa Host và EFR32 NCP

Giao tiếp serial là thành phần đóng vai trò nền tảng trong kiến trúc Host–NCP (Network Co-Processor) của hệ thống IoT. Trong mô hình này, vi điều khiển trung tâm EFR32 đóng vai trò bộ xử lý phụ trách toàn bộ Zigbee stack ở mức thấp, trong khi Host là máy tính Linux chạy ứng dụng Gateway xử lý logic nghiệp vụ và kết nối mạng ở mức cao. Để liên kết hai thành phần phần cứng và phần mềm riêng biệt này, Silicon Labs thiết lập một mô hình phân lớp giao thức chặt chẽ, bao gồm lớp vật lý UART, lớp liên kết dữ liệu ASH và lớp ứng dụng EZSP [12, 13].

Lớp vật lý sử dụng giao diện UART (Universal Asynchronous Receiver/Transmitter) để truyền nhận dữ liệu nối tiếp bất đồng bộ [11]. Cấu hình UART tiêu chuẩn cho giao tiếp này sử dụng tốc độ Baud rate là 115200 bps, độ dài khung dữ liệu 8 bit, không sử dụng bit parity, và sử dụng 1 stop bit (cấu hình 8-N-1). Điểm đặc biệt quan trọng trong lớp vật lý này là việc bắt buộc phải sử dụng cơ chế kiểm soát luồng phần cứng RTS/CTS (Request to Send / Clear to Send). Cơ chế kiểm soát luồng giúp ngăn ngừa hiện tượng mất mát dữ liệu khi bộ đệm serial của chip EFR32 NCP hoặc của Host Linux bị tràn do tốc độ xử lý không đồng đều giữa hai bên.

Do giao tiếp UART thuần túy là bất đồng bộ và dễ bị ảnh hưởng bởi nhiễu môi trường vật lý, Silicon Labs đã thiết kế giao thức liên kết dữ liệu ASH (Asynchronous Serial Host) để chạy trực tiếp trên đường truyền UART [12, 13]. Giao thức ASH đảm nhận nhiệm vụ định khung dữ

liệu bằng cách sử dụng byte cờ (flag byte) cố định là 0x7E để đánh dấu điểm bắt đầu và điểm kết thúc của mỗi khung truyền. Để bảo toàn tính toàn vẹn của dữ liệu trước ảnh hưởng của nhiễu đường truyền, mỗi khung ASH đều đi kèm trường CRC (Cyclic Redundancy Check) 16 bit được tính toán tự động ở cuối khung.

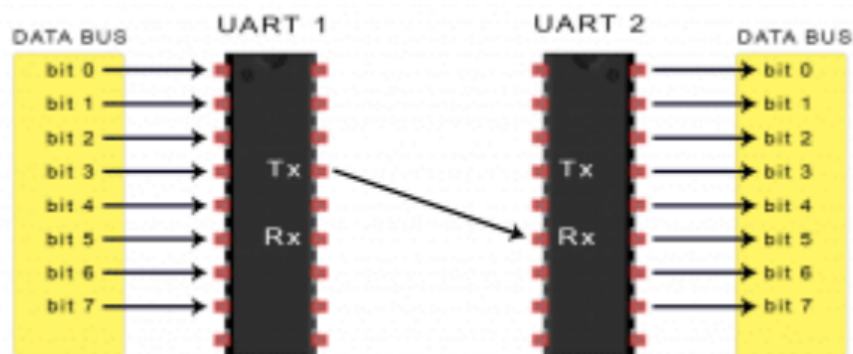
Bên cạnh khả năng phát hiện lỗi bằng CRC, giao thức ASH còn tích hợp cơ chế kiểm soát luồng và sửa lỗi thông qua kỹ thuật cửa sổ trượt (sliding window) cùng các khung điều khiển xác nhận ACK (Acknowledgement) và báo lỗi NAK (Negative Acknowledgement). Khi Host hoặc NCP phát hiện khung dữ liệu bị lỗi CRC hoặc mất gói tin do lệch số thứ tự khung, bên nhận sẽ gửi khung NAK yêu cầu bên gửi truyền lại khung đó từ bộ đệm. Để khởi động hoặc khôi phục lại kết nối serial sau khi khởi động lại hệ thống, Host sẽ gửi khung điều khiển RST (Reset) xuống NCP, và NCP phản hồi lại bằng khung RSTACK (Reset Acknowledge) để đưa trạng thái truyền nhận của cả hai bên về điểm xuất phát đồng bộ.

Một kỹ thuật đặc trưng khác của lớp liên kết dữ liệu ASH là chèn byte (byte stuffing) để đảm bảo tính minh bạch nhị phân của dữ liệu. Khi payload của khung dữ liệu chứa các byte có giá trị trùng với các byte điều khiển đặc biệt của ASH như cờ khởi đầu 0x7E, byte thoát 0x7D, hoặc các byte kiểm soát luồng phần mềm XON/XOFF (0x11/0x13), ASH sẽ tự động chèn thêm byte thoát 0x7D phía trước và biến đổi byte dữ liệu gốc bằng cách thực hiện phép toán XOR với giá trị mặt nạ 0x20 (còn gọi là ASH_FLIP). Quá trình này được thực hiện đảo ngược ở bên nhận, giúp ngăn ngừa việc hệ thống nhận diện nhầm dữ liệu ngẫu nhiên thành lệnh điều khiển khung vật lý.

Nằm ở đỉnh của mô hình phân lớp giao tiếp này là giao thức ứng dụng EZSP (EmberZNet Serial Protocol) do Silicon Labs phát triển [12]. EZSP định nghĩa một tập lệnh chuẩn hóa cho phép ứng dụng Host có thể cấu hình, truy vấn trạng thái và ra lệnh cho stack Zigbee EmberZNet đang chạy trên chip EFR32 NCP. Giao thức hoạt động chủ yếu theo mô hình lệnh–phản hồi (command-response), nơi Host luôn là bên chủ động gửi yêu cầu (Request) và NCP trả về kết quả (Response) tương ứng sau khi xử lý.

Tuy nhiên, do các sự kiện trong mạng Zigbee (như thiết bị mới tham gia mạng hoặc có thông điệp báo cáo trạng thái gửi về) xảy ra một cách ngẫu nhiên, EZSP hỗ trợ cơ chế hàm gọi ngược (callback) để NCP chủ động thông báo cho Host. Các thông báo này được NCP xếp vào hàng đợi cục bộ và gửi ngược lên Host khi nhận được lệnh truy vấn hoặc gửi xen kẽ trong các khung phản hồi thông thường. Mỗi khung EZSP bao gồm số thứ tự khung (Sequence Number), byte điều khiển khung (Frame Control) xác định loại thông điệp, mã định danh lệnh (Command ID) và các tham số dữ liệu cụ thể đi kèm.

Mô hình phân lớp UART, ASH và EZSP mang lại lợi ích to lớn về mặt kiến trúc hệ thống khi tách biệt hoàn toàn ứng dụng Gateway trên Linux Host khỏi các ràng buộc thời gian thực khắt khe của stack giao thức Zigbee. Việc xử lý truyền nhận tần số vô tuyến, quản lý bảng định tuyến mesh phức tạp và mã hóa bảo mật lớp mạng được thực thi trọn vẹn trên chip EFR32 NCP. Nhờ đó, ứng dụng Gateway Host Linux có thể tập trung tài nguyên cho các tác vụ xử lý cấp cao như tích hợp giao thức MQTT, đồng bộ dữ liệu đám mây Cloud, quản lý cơ sở dữ liệu và vận hành các kịch bản tự động hóa cục bộ một cách hiệu quả.



Hình 2.5: Minh họa giao tiếp UART giữa hai thành phần phần cứng

Hình 2.5 minh họa cấu trúc giao tiếp vật lý giữa hai thành phần phần cứng thông qua các đường tín hiệu TX, RX. Trong kiến trúc Host–NCP của hệ thống, đường vật lý UART này tương ứng trực tiếp với kết nối giữa Gateway Host Linux và chip EFR32 NCP, nơi các giao thức EZSP/ASH được sử dụng để truyền nhận các lệnh Zigbee. Điểm cần phân biệt là UART/EZSP/ASH thuộc biên giao tiếp giữa Host và NCP, còn application contract của hệ thống nằm ở MQTT và Zigbee/ZCL action mapping. Do đó, các giao tiếp ở mức truyền serial vật lý này không nên được nhầm lẫn với các API điều khiển logic đầu cuối của đồ án. Giao tiếp serial này cũng là thành phần chính cần giám sát chặt chẽ bằng nhật ký hệ thống trong trường hợp xảy ra sự cố mất kết nối vật lý hoặc mất khung truyền.

2.10 REST API, Cloud Backend và Database

Cloud Backend là lớp phần mềm trung tâm ở phía server. Trong hệ thống Smart Building, Cloud Backend cung cấp REST API cho App, nhận trạng thái từ Gateway, lưu dữ liệu vào database và gửi command xuống Gateway khi người dùng hoặc rule automation yêu cầu. Nhờ có Backend, App không cần làm việc trực tiếp với Zigbee, cluster hoặc NCP.

Representational State Transfer (REST) là phong cách thiết kế API phổ biến trên nền HTTP. App có thể gọi API để lấy danh sách thiết bị, trạng thái hiện tại, lịch sử event hoặc gửi yêu cầu điều khiển. REST dễ triển khai và dễ kiểm thử, phù hợp với các thao tác quản lý thông thường. Hạn chế của REST là mô hình request/response không tối ưu cho cập nhật realtime liên tục nếu dùng một mình; với yêu cầu realtime cao hơn, hệ thống có thể cần thêm WebSocket hoặc cơ chế push.

Database lưu trữ các dữ liệu bền vững của hệ thống. Các nhóm dữ liệu thường gặp gồm device registry, device state, event history, command lifecycle và automation rule. Device registry mô tả thiết bị trong hệ thống. Device state biểu diễn trạng thái hiện tại hoặc lịch sử trạng thái. Event history hỗ trợ truy vết sự kiện. Command lifecycle cho biết lệnh đã được tạo, gửi, xác nhận hay thất bại. Automation rule mô tả điều kiện và hành động tự động.

Trong đồ án, FastAPI được sử dụng cho REST API, SQLAlchemy hỗ trợ Object-Relational Mapping (ORM), PostgreSQL dùng cho dữ liệu production và Paho MQTT giúp Cloud kết nối với broker. Ở Chương 2, nhóm thực hiện chỉ trình bày vai trò của các thành phần này ở mức công nghệ nền; danh sách endpoint, schema chi tiết và xử lý nghiệp vụ được đặt ở các chương sau.

Bảng 2.4: Vai trò của các thành phần Cloud trong hệ thống

Thành phần	Vai trò lý thuyết
REST API	Cung cấp giao diện HTTP để App lấy dữ liệu và gửi yêu cầu điều khiển.
MQTT Client	Subscribe trạng thái từ Gateway và publish command xuống Gateway.
Database	Lưu thiết bị, trạng thái, sự kiện, command và automation rule.
ORM	Ảnh xạ giữa object trong code backend và bảng dữ liệu trong database.

Bảng 2.4 phân định rõ vai trò của từng thành phần trong lớp Cloud. Nhờ sự phân công trách nhiệm này, mỗi thành phần có thể được kiểm thử và thay thế độc lập mà không làm ảnh hưởng đến các thành phần còn lại. Chi tiết về các endpoint REST API, MQTT topic và cấu trúc cơ sở dữ liệu được trình bày cụ thể trong các chương thiết kế và triển khai.

2.11 Mobile App và State Management

Mobile App là lớp tương tác trực tiếp với người dùng cuối. Trong hệ thống Smart Building, người dùng cần xem trạng thái thiết bị, gửi lệnh điều khiển và theo dõi các event hoặc automation rule. App vì vậy chuyển thao tác của người dùng thành request gửi lên Cloud Backend, đồng thời trình bày lại dữ liệu mà Backend cung cấp.

Flutter là framework UI cross-platform, cho phép xây dựng giao diện trên nhiều nền tảng từ cùng một codebase [14]. Trong đồ án, App gọi REST API qua HTTP. Cách tổ chức này giữ App ở lớp người dùng và Cloud, không để App giao tiếp trực tiếp với Zigbee hoặc Gateway. Nhờ đó, App tập trung vào giao diện và trải nghiệm sử dụng, còn Gateway và Cloud xử lý các phần truyền thông thiết bị.

State management là cách App quản lý và cập nhật trạng thái giao diện. Khi danh sách thiết bị, trạng thái đèn hoặc log automation thay đổi, giao diện cần được cập nhật nhất quán. Provider là một cơ chế state management phổ biến trong Flutter, giúp tách dữ liệu/trạng thái khỏi widget hiển thị. Với đồ án này, state management giúp App phản ánh đúng trạng thái thiết bị sau khi người dùng gửi lệnh hoặc khi Gateway báo trạng thái mới.

Ưu điểm của việc để App làm việc qua REST API là kiến trúc rõ ràng và dễ kiểm thử. Hạn chế là cập nhật realtime có thể chưa mượt nếu chỉ dựa vào request/response. Vì vậy, trong phạm vi Chương 2, App được trình bày như lớp monitoring/control cho người dùng; chi tiết màn hình và luồng thao tác được trình bày ở chương thiết kế hoặc triển khai.

2.12 Firmware, Bootloader và OTA

Firmware là phần mềm chạy trực tiếp trên thiết bị nhúng. Khác với ứng dụng mobile hoặc backend chạy trên hệ điều hành tổng quát, firmware làm việc gần hơn với phần cứng như radio, GPIO, cảm biến và bộ nhớ flash. Trong hệ thống Zigbee, firmware quyết định thiết bị hoạt động như light, switch, occupancy sensor hay NCP coordinator.

Bootloader là chương trình nhỏ chạy trước firmware chính, hỗ trợ nạp, kiểm tra hoặc cập nhật firmware [?]. Trong quá trình phát triển, firmware artifact như file .s37 có thể được dùng để flash thiết bị bằng công cụ của Silicon Labs. Artifact không phải là chức năng mà người dùng cuối thao tác trực tiếp, nhưng là đầu ra cần thiết khi build và nạp firmware cho board.

Over-the-Air (OTA) là cơ chế cập nhật firmware qua mạng thay vì flash thủ công qua máy tính [15]. Với Zigbee OTA, hệ thống thường cần OTA image, quản lý version, kiểm tra tính toàn vẹn, OTA server/client và quá trình truyền firmware theo từng phần. OTA có ý nghĩa khi số lượng thiết bị tăng lên hoặc thiết bị được lắp ở vị trí khó tiếp cận.

OTA cũng làm hệ thống phức tạp hơn vì liên quan đến phân phối firmware, xử lý lỗi cập nhật, kiểm tra version, bảo mật image và khả năng khôi phục khi cập nhật thất bại. Do đó, trong báo cáo này, OTA được trình bày chủ yếu như nền tảng công nghệ và hướng phát triển. Nhóm thực hiện không khẳng định OTA end-to-end đã hoàn chỉnh nếu chưa có bằng chứng triển khai đầy đủ trong các chương sau.

2.13 Security trong Zigbee và IoT

Security trong hệ thống IoT cần được xem xét ở nhiều lớp, từ thiết bị, mạng Zigbee, Gateway, MQTT Broker, Cloud Backend, Database đến App. Với Smart Building, sự cố bảo mật có thể dẫn đến mất dữ liệu hoặc làm sai lệch hành vi điều khiển thiết bị vật lý. Vì vậy, báo cáo cần nêu các nguyên tắc bảo mật cơ bản trước khi đi vào phần thiết kế hệ thống.

Ở lớp Zigbee, các khái niệm cần chú ý gồm network key, application security và Trust Center. Network key giúp bảo vệ truyền thông trong cùng một mạng Zigbee. Trust Center liên quan đến việc quản lý thiết bị tham gia mạng và phân phối khóa trong một số mô hình bảo mật. Khi thiết bị join mạng, hệ thống cần kiểm soát thiết bị nào được phép tham gia và quá trình trao đổi khóa được thực hiện như thế nào.

Ở lớp MQTT và Cloud, broker cần có authentication, phân quyền topic và trong môi trường production nên cân nhắc Transport Layer Security (TLS). Cloud Backend cần kiểm soát quyền truy cập API, kiểm tra dữ liệu đầu vào và bảo vệ database. App cũng cần xử lý token hoặc thông tin đăng nhập cẩn thận để tránh người dùng không hợp lệ gửi command điều khiển thiết bị.

Firmware security là lớp bảo mật quan trọng khác. Firmware signing giúp giảm rủi ro thiết bị nhận firmware giả mạo. Với OTA, yêu cầu bảo mật còn cao hơn vì firmware được truyền qua mạng. Tuy nhiên, các cơ chế như install-code Trust Center join, APS encryption hardening, GBL signing hoặc MQTT mTLS chỉ được xem là nền tảng hoặc hướng phát triển nếu chưa có bằng chứng hoàn thiện trong hệ thống hiện tại.

Việc tăng cường security giúp hệ thống an toàn hơn nhưng cũng làm quá trình triển khai, vận hành và debug phức tạp hơn. Vì vậy, đồ án cần trình bày đúng phạm vi: nêu nguyên tắc bảo mật nền tảng, chỉ khẳng định những phần có evidence, và đưa các cơ chế nâng cao vào hướng phát triển khi chưa triển khai đầy đủ.

2.14 Công nghệ sử dụng trong đồ án

Các công nghệ trong đồ án được tổ chức theo kiến trúc nhiều lớp. Thiết bị Zigbee nằm ở tầng nhúng và mạng cục bộ. Gateway chịu trách nhiệm chuyển đổi giữa Zigbee và MQTT. MQTT Broker làm trung gian messaging. Cloud Backend quản lý dữ liệu và API. Mobile App là giao diện cho người dùng cuối. Cách chia lớp này giúp hệ thống dễ giải thích, dễ kiểm thử theo từng phần và thuận lợi hơn khi mở rộng.

Bảng 2.5: Tóm tắt công nghệ nền được sử dụng trong đồ án

Lớp	Công nghệ	Vai trò trong hệ thống
Device	EFR32, EmberZNet/Gecko SDK, Zigbee stack	Chạy firmware cho light, switch, occupancy sensor hoặc NCP.
Gateway	Z3GatewayHost, C, libmosquitto	Kết nối Zigbee local network với MQTT/Cloud.
Broker	Eclipse Mosquitto	Trung gian publish/subscribe giữa Cloud và Gateway.
Backend	FastAPI, SQLAlchemy, Paho MQTT	Cung cấp REST API, xử lý MQTT và quản lý dữ liệu/lệnh.
Database	PostgreSQL, SQLite trong một số ngữ cảnh test	Lưu thiết bị, trạng thái, sự kiện, command và automation.
App	Flutter, Dart, Provider, HTTP	Giao diện monitoring/control cho người dùng cuối.

Tóm lại, Chương 2 cung cấp phần cơ sở để người đọc hiểu các lựa chọn kỹ thuật chính của đồ án: Zigbee cho mạng thiết bị, Z3Gateway C cho lớp kết nối với coordinator radio, MQTT cho trao đổi trạng thái/lệnh, REST API và database cho lớp Cloud, Flutter cho giao diện người dùng. Các khái niệm này là tiền đề cho phân phân tích yêu cầu, thiết kế kiến trúc, triển khai và đánh giá ở các chương tiếp theo.

CHƯƠNG 3

PHÂN TÍCH YÊU CẦU VÀ THIẾT KẾ HỆ THỐNG

Sau phần cơ sở lý thuyết, chương này chuyển sang mô tả hệ thống trong phạm vi đồ án. Hiện tại, hệ thống được triển khai theo tuyến điều khiển Cloud-to-Gateway. Ứng dụng di động Flutter gửi yêu cầu qua REST API tới Cloud Backend FastAPI. Cloud lưu dữ liệu bằng PostgreSQL, phát lệnh qua MQTT Broker Mosquitto, sau đó Gateway Z3GatewayHost viết bằng C nhận lệnh bằng thư viện libmosquitto và chuyển xuống mạng Zigbee thông qua EFR32 NCP. Các thiết bị Zigbee cuối gồm đèn, công tắc và cảm biến chuyển động.

3.1 Phạm vi và giả định thiết kế

Phần thiết kế của đồ án tập trung vào một hệ thống Smart Building quy mô nhỏ. Hệ thống cần cho phép người dùng theo dõi trạng thái thiết bị, điều khiển đèn, xem sự kiện và cấu hình một phần automation rule. Các chức năng được mô tả trong chương này bám theo những thành phần đã có trong mã nguồn, không mở rộng sang các phần chưa có bằng chứng triển khai.

Một số khái niệm cơ bản:

- **Contract (hợp đồng dữ liệu):** quy ước giữa các module, ví dụ topic MQTT, cấu trúc JSON, REST request và response.
- **Reported state (trạng thái báo cáo):** trạng thái do thiết bị hoặc Gateway gửi lên Cloud, ví dụ đèn đang bật, mức sáng hiện tại, thiết bị còn reachable hay không.
- **Command lifecycle (vòng đời lệnh):** các bước một lệnh đi qua từ lúc Mobile tạo lệnh, Cloud lưu lệnh, Gateway nhận lệnh, gửi xuống Zigbee và trả kết quả.

Bảng 3.1 tóm tắt phạm vi được dùng khi viết chương này.

Bảng 3.1: Phạm vi thiết kế trong đồ án

Nhóm chức năng	Được mô tả trong chương	Không khẳng định trong phạm vi hiện tại
Điều khiển thiết bị	Điều khiển đèn qua Mobile, REST API, MQTT và Gateway Zigbee.	Không xem công tắc và cảm biến chuyển động là thiết bị có thể nhận lệnh trực tiếp.
Theo dõi trạng thái	Lưu và hiển thị reported state, registry, event và gateway health.	Không khẳng định mọi loại telemetry đều đã hoàn chỉnh cho sản phẩm thương mại.
Automation	Cloud có CRUD automation rule và publish desired automation. Gateway có rule engine local cho switch-to-light khi bật cấu hình môi trường.	Không khẳng định Cloud-created automation đã được Gateway tự động áp dụng end-to-end.
Bảo mật	MQTT Broker có username/password và ACL.	Không khẳng định đã có mTLS, multi-user auth hoàn chỉnh hoặc install-code commissioning.

Bảng 3.1 giúp xác định rõ ranh giới mô tả trong chương, tránh việc đưa vào báo cáo các chức năng chưa có bằng chứng triển khai hoàn chỉnh. Các giới hạn được ghi nhận rõ ràng để người đọc hiểu đúng phạm vi của hệ thống trong giai đoạn đồ án.

3.2 Phân tích yêu cầu hệ thống

3.2.1 Tác nhân sử dụng hệ thống

Hệ thống có ba nhóm tác nhân chính. Người dùng thao tác qua Mobile App. Cloud Backend tiếp nhận request, lưu dữ liệu và làm cầu nối với MQTT Broker. Gateway là tác nhân kỹ thuật nằm gần mạng Zigbee, chịu trách nhiệm nhận lệnh MQTT và phát lệnh xuống thiết bị.

Bảng 3.2: Các tác nhân chính trong hệ thống

Tác nhân	Vai trò	Giao tiếp chính
Người dùng Mobile	Xem dashboard, xem danh sách thiết bị, điều khiển đèn, xem event log và thao tác automation rule.	REST API tới Cloud Backend.
Cloud Backend	Quản lý thiết bị, trạng thái, lệnh, sự kiện và automation. Thành phần này cũng publish/subscribe MQTT để trao đổi với Gateway.	REST, PostgreSQL, MQTT.
Gateway Z3GatewayHost	Nhận MQTT command, đưa lệnh vào queue, xử lý trong vòng lặp chính và gọi Zigbee stack để điều khiển thiết bị.	MQTT qua libmosquitto, Zigbee qua EFR32 NCP.
Thiết bị Zigbee	Đèn báo cáo trạng thái và nhận lệnh On/Off. Công tắc phát sự kiện toggle. Cảm biến chuyển động báo occupancy.	Zigbee Cluster Library.

Bảng 3.2 làm rõ sự phân công trách nhiệm giữa ba nhóm tác nhân chính trong hệ thống. Nhờ việc phân định rõ giao tiếp chính của từng tác nhân, phần thiết kế API, MQTT contract và cấu trúc cơ sở dữ liệu sẽ được xây dựng theo đúng vòng đời dữ liệu của hệ thống.

3.2.2 Yêu cầu chức năng

Các yêu cầu chức năng được rút ra từ đường chạy hiện tại của hệ thống. Bảng 3.3 dùng mã FR để thuận tiện đối chiếu khi thiết kế API, cơ sở dữ liệu và MQTT contract.

Bảng 3.3: Các yêu cầu chức năng chính

Mã	Yêu cầu	Ghi chú thiết kế
FR-01	Người dùng xem được danh sách thiết bị, thông tin phòng và trạng thái mới nhất của từng thiết bị.	Dữ liệu lấy từ Cloud qua REST API.
FR-02	Người dùng điều khiển bật/tắt đèn từ Mobile App.	Lệnh đi qua Cloud, MQTT Broker, Gateway rồi tới mạng Zigbee.
FR-03	Gateway gửi reported state, registry, event và health lên Cloud.	Các bản tin uplink đi qua MQTT.
FR-04	Cloud lưu command, state, event và automation rule để Mobile truy vấn lại.	PostgreSQL là nơi lưu trữ chính.
FR-05	Người dùng xem được event log và trạng thái xử lý command.	Mobile có cơ chế polling command sau khi gửi lệnh.
FR-06	Người dùng tạo, sửa, bật/tắt và xóa automation rule ở phía Cloud.	Rule được lưu ở Cloud và publish lên desired automation topic; Gateway hiện chưa tự động consume topic này.
FR-07	Hệ thống hỗ trợ mở/đóng commissioning và rediscover thiết bị qua Gateway API.	Đây là lệnh Gateway-level, không phải lệnh điều khiển trực tiếp một thiết bị cụ thể.

Bảng 3.3 liệt kê đầy đủ các yêu cầu chức năng cần thực hiện trong phạm vi đồ án. Mã FR giúp việc đối chiếu yêu cầu với endpoint API, MQTT topic tương ứng và các kịch bản kiểm thử ở các chương sau được thuận tiện hơn.

3.2.3 Yêu cầu phi chức năng

Yêu cầu phi chức năng trong đồ án được đặt ở mức phù hợp với hệ thống thử nghiệm. Các tiêu chí này dùng để định hướng thiết kế, không được hiểu là số đo đã kiểm chứng trên môi trường sản xuất.

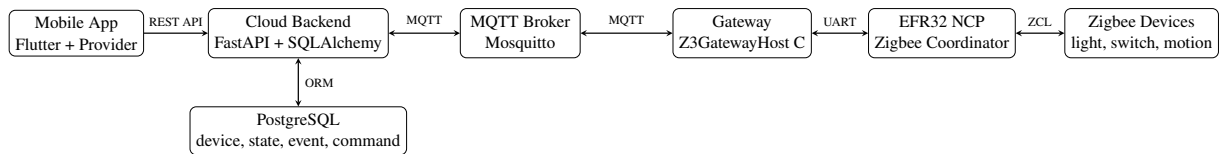
Bảng 3.4: Các yêu cầu phi chức năng

Tiêu chí	Yêu cầu thiết kế	Cách đáp ứng trong hệ thống
Độ trễ thao tác	Lệnh bật/tắt đèn cần đi qua ít tầng trung gian nhất có thể trong mô hình Cloud-to-Gateway.	Mobile gửi REST command, Cloud publish MQTT, Gateway xử lý queue và gọi Zigbee stack.
Tính nhất quán dữ liệu	Trạng thái hiển thị trên Mobile phải dựa trên reported state từ Gateway thay vì chỉ dựa vào thao tác người dùng.	Gateway publish reported state retained; Cloud lưu device state.
Khả năng quan sát	Người vận hành cần biết Gateway online, health và command reply.	Gateway publish online, health, event và command reply qua MQTT.
An toàn truy cập	MQTT Broker cần tách quyền publish/subscribe theo cấu hình broker.	Mosquitto dùng username/password và ACL.
Để bảo trì	Các tầng Mobile, Cloud, Broker, Gateway và firmware nên tách trách nhiệm rõ ràng.	Cloud dùng FastAPI router; Gateway tách MQTT, parser, dispatcher, light/switch logic và rule engine.

Bảng 3.4 đặt ra các tiêu chí định hướng thiết kế cho từng lớp chức năng của hệ thống. Các yêu cầu này được sử dụng như cơ sở để lựa chọn giải pháp kỹ thuật và đánh giá hiệu năng trong quá trình kiểm thử.

3.3 Kiến trúc tổng thể hệ thống

Kiến trúc runtime hiện tại gồm năm lớp: Mobile App, Cloud Backend, MQTT Broker, Gateway và mạng Zigbee. Docker compose production chứa các service Cloud API, Mosquitto và PostgreSQL. Mobile App và Gateway không nằm trong compose này; chúng chạy như thành phần riêng và kết nối vào Cloud/Broker qua network.



Hình 3.1: Kiến trúc runtime của hệ thống Smart Building trong đồ án

Trong kiến trúc này, Cloud không điều khiển trực tiếp thiết bị Zigbee. Cloud tạo command và phát bản tin MQTT. Gateway mới là nơi hiểu phần Zigbee runtime, ví dụ mapping command sang cluster On/Off, đẩy lệnh xuống EFR32 NCP và gửi command reply ngược lại Cloud.

Bảng 3.5: Trách nhiệm của các khối chính

Khối	Trách nhiệm chính	Dữ liệu ra/vào
Mobile App	Giao diện thao tác thiết bị, event, automation và settings.	REST request/response.
Cloud Backend	Xử lý REST API, lưu dữ liệu, publish command, subscribe reported state, event, registry và health.	JSON qua REST, MQTT envelope, bản ghi PostgreSQL.
MQTT Broker	Trung gian truyền bản tin giữa Cloud và Gateway.	Topic prefix: sb/v1/{tenant}/ {site}/ {gateway}.
Gateway Host	Nhận command, parse contract, route lệnh tới module phù hợp, publish reply và telemetry.	MQTT, command queue, Zigbee API.
Zigbee Devices	Thực hiện lệnh và báo cáo trạng thái/sự kiện.	ZCL command, ZCL report.

3.4 Thiết kế thiết bị và trạng thái

3.4.1 Device model

Device model của hệ thống hiện tại dùng các loại thiết bị light, switch, motion và unknown. Điểm dễ nhầm là firmware cảm biến có tên Occupancy Sensor, nhưng khi đi qua Gateway/Cloud/Mobile, loại thiết bị được chuẩn hóa là motion. Vì vậy trong chương này không dùng occupancy_sensor như một device type runtime.

Bảng 3.6: Ánh xạ loại thiết bị và chức năng runtime

Device type	Vai trò	State/Event chính	Command hỗ trợ
light	Thiết bị chấp hành, nhận lệnh bật/tắt.	power, level, reachable.	on, off end-to-end. set_level có trong schema nhưng Gateway hiện chưa xử lý hoàn chỉnh.
switch	Thiết bị phát sự kiện điều khiển cục bộ.	Event toggle; schema có thể lưu reachable, battery.	Không commandable trong Gateway hiện tại.
motion	Cảm biến chuyển động/occupancy.	occupancy với trạng thái occupied hoặc unoccupied khi có event phù hợp.	Không có lệnh điều khiển trực tiếp.
unknown	Thiết bị chưa phân loại được khi discovery.	Registry metadata nếu có.	Không dùng cho điều khiển.

Bảng 3.6 mô tả rõ từng loại thiết bị được hệ thống hỗ trợ cùng với truy trạng thái và khả năng nhận lệnh tương ứng. Việc chỉ rõ loại thiết bị nào có thể commandable giúp tránh thiết kế sai lệnh điều khiển tới các thiết bị chưa được hỗ trợ như switch hoặc cảm biến.

3.4.2 State, event và command

Hệ thống tách ba loại dữ liệu để tránh lẫn giữa trạng thái hiện tại, sự kiện tức thời và yêu cầu điều khiển:

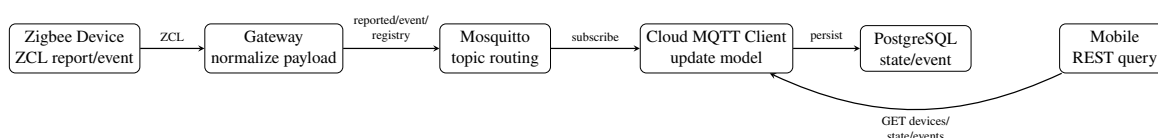
- **State:** ảnh chụp trạng thái gần nhất của thiết bị. Ví dụ đèn có power=on, level=100, reachable=true.
- **Event:** sự kiện xảy ra tại một thời điểm. Ví dụ công tắc phát toggle hoặc cảm biến báo occupancy_changed.
- **Command:** yêu cầu điều khiển do người dùng hoặc hệ thống tạo ra. Ví dụ Mobile yêu cầu bật đèn.

Cách tách này giúp Cloud không phải suy đoán trạng thái thiết bị chỉ từ thao tác trên Mobile. Sau khi gửi command, Mobile vẫn cần lấy lại command status hoặc chờ reported state mới để hiển thị đúng hơn.

3.5 Luồng dữ liệu chính

3.5.1 Luồng uplink: thiết bị báo cáo lên Cloud

Uplink là chiều dữ liệu từ mạng Zigbee đi lên Cloud. Đèn gửi ZCL report, công tắc gửi event, cảm biến motion gửi thay đổi occupancy. Gateway nhận các thông tin này, chuẩn hóa thành MQTT payload và publish lên Broker. Cloud subscribe các topic tương ứng, cập nhật database và cung cấp lại dữ liệu cho Mobile qua REST API.

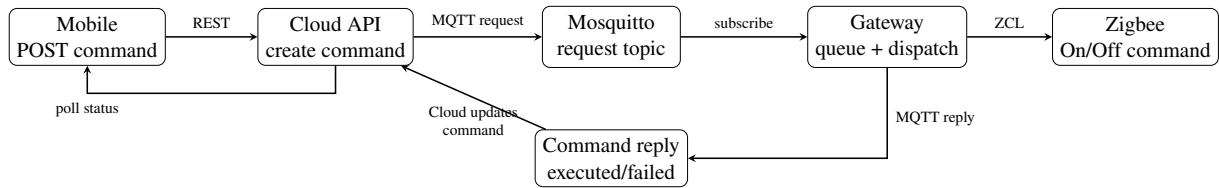


Hình 3.2: Luồng uplink đồng bộ trạng thái và sự kiện

Các bản tin reported state, registry và gateway health được thiết kế theo hướng giữ lại bản tin mới nhất trên Broker bằng retain flag ở phía Gateway. Với event và command reply, hệ thống không dùng retained message vì đây là dữ liệu theo thời điểm.

3.5.2 Luồng downlink: người dùng gửi lệnh điều khiển

Downlink là chiều điều khiển từ Mobile xuống thiết bị. Người dùng bấm bật/tắt đèn trên Mobile. Mobile gọi REST API tạo command. Cloud lưu command, chuyển command thành MQTT envelope và publish tới topic request của Gateway. Gateway nhận bản tin, đưa vào queue, xử lý trong main tick, gọi module light control rồi gửi lệnh Zigbee On/Off xuống thiết bị.

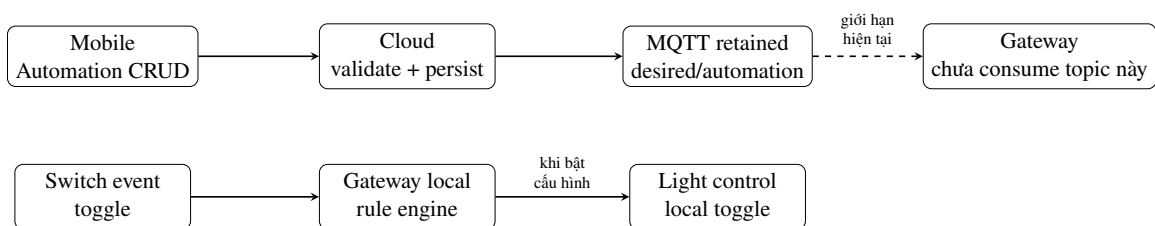


Hình 3.3: Luồng downlink và phản hồi command

Trong phần đánh giá, cần hiểu đúng ý nghĩa của trạng thái executed. Với light command hiện tại, trạng thái này phản ánh việc Gateway đã gửi lệnh xuống Zigbee ở mức truyền lệnh, chưa phải một xác nhận ứng dụng đầy đủ rằng thiết bị vật lý chắc chắn đã đổi trạng thái. Vì vậy reported state sau đó vẫn là nguồn đáng tin hơn để hiển thị trạng thái thiết bị.

3.5.3 Luồng automation

Automation hiện có hai phần cần phân biệt. Phía Cloud đã có API tạo/sửa/xóa automation rule, validate trigger/action và publish desired automation rule lên MQTT retained topic. Phía Gateway có rule engine local cho tình huống switch-to-light, nhưng mặc định không có binding nếu chưa bật biến môi trường SB_RULES_SWITCH_TO_LIGHT=1. Trong mã nguồn hiện tại chưa thấy Gateway subscribe topic desired/automation.



Hình 3.4: Ranh giới thiết kế automation trong phiên bản hiện tại

Thiết kế này cho phép báo cáo rõ phần đã làm và phần còn mở. Cloud đã có nền tảng quản lý rule. Gateway đã có hướng xử lý local rule. Bước còn thiếu để có dynamic cloud-synced automation là cơ chế Gateway nhận desired automation, đồng bộ rule và chạy rule đó theo event thực tế.

3.6 Thiết kế MQTT contract

MQTT contract dùng prefix chung:

sb/v1/{tenant}/{site}/{gateway}

Trong môi trường thử nghiệm, các giá trị tenant, site và gateway có thể được cấu hình theo lab. Khi viết báo cáo, nên trình bày theo biến như trên để tránh khóa thiết kế vào một máy cụ thể.

Bảng 3.7: Các nhóm topic MQTT chính

Chiều	Topic	Payload chính	QoS/retain
Cloud tới Gateway	.../commands/{command_id}/request	Envelope có schema, msg_id, correlation_id và payload command.	Cloud publish QoS 0; Gateway subscribe QoS 1.
Gateway tới Cloud	.../commands/{command_id}/reply	command_id, device_id, status, reason.	QoS 1, không retain.
Gateway tới Cloud	.../devices/{device_type}/{device_id}/reported	Reported state của thiết bị.	QoS 1, retain.
Gateway tới Cloud	.../devices/{device_type}/{device_id}/event	Event như toggle hoặc occupancy_changed.	QoS 1, không retain.
Gateway tới Cloud	.../devices/{device_type}/{device_id}/registry	Identity, endpoint, cluster và metadata.	QoS 1, retain.
Gateway tới Cloud	.../gateway/online .../gateway/health	Online/offline, uptime, mqtt_connected, known_device_count.	QoS 1, retain.
Cloud tới Broker	.../desired/automation/{automation_id}	Automation payload và cờ deleted.	QoS 0, retain.

Envelope command request được thiết kế để Gateway có đủ thông tin định danh và correlation khi trả kết quả. Ví dụ rút gọn:

Mã nguồn 3.1: Ví dụ MQTT command envelope

```
{
  "schema": "sb.v1.command",
  "msg_id": "msg-001",
  "ts": "2026-05-27T10:00:00Z",
  "tenant_id": "hust",
  "site_id": "lab01",
  "gateway_id": "gw-ubuntu-01",
  "source": "cloud-api",
  "correlation_id": "command-001",
  "payload": {
    "op": "device.command",
    "device_id": "light-001",
    "cluster": "0x0006",
    "command": "on"
  }
}
```

Reported state của đèn có dạng:

Mã nguồn 3.2: Ví dụ reported state của thiết bị đèn

```
{
  "device_id": "light-001",
```

```

"device_type": "light",
"eui64": "00124b...",
"nwk_addr": "0x1234",
"state": {
  "power": "on",
  "level": 100,
  "reachable": true
}
}

```

3.7 Thiết kế Cloud Backend và cơ sở dữ liệu

3.7.1 REST API

Cloud Backend cung cấp REST API cho Mobile App. API được chia theo nhóm tài nguyên thay vì đưa toàn bộ logic vào một endpoint chung. Cách chia này phù hợp với FastAPI router và giúp Mobile chỉ gọi đúng nhóm dữ liệu cần dùng.

Bảng 3.8: Các nhóm REST API chính

Nhóm API	Endpoint tiêu biểu	Mục đích
Health	GET /health	Kiểm tra Cloud API đang chạy.
Devices	GET /api/devices, GET /api/devices/{id}, GET /api/devices/{id}/state	Lấy danh sách thiết bị và state mới nhất.
Commands	POST /api/devices/{id}/command GET /api/commands/{id}	Tạo lệnh điều khiển và truy vấn trạng thái lệnh.
Events	GET /api/events	Lọc và xem event theo thiết bị hoặc loại event.
Automations	/api/automations /api/automation-events	CRUD automation rule, bật/tắt rule và xem automation event.
Gateways	/api/gateways/{id}/commissioning/open /api/devices/{id}/rediscover	Gửi lệnh commissioning và rediscover tới Gateway.

Mobile App có login UI nhưng trong Cloud Backend hiện tại chưa có auth router tương ứng. Vì vậy, chương này không mô tả multi-user authentication như một chức năng đã hoàn thiện.

3.7.2 Cơ sở dữ liệu

Cloud Backend dùng ORM model làm nguồn thiết kế chính cho cơ sở dữ liệu. Khi service khởi động, hệ thống gọi cơ chế tạo bảng từ model. Các bảng chính được mô tả trong Bảng 3.9.

Bảng 3.9: Các bảng dữ liệu chính

Bảng	Vai trò	Liên quan tới
homes, rooms	Lưu cấu trúc không gian để gán thiết bị vào khu vực sử dụng.	Device list, dashboard.
users	Bảng người dùng đã có trong model.	Chưa có auth flow hoàn chỉnh trong Cloud API.
devices	Lưu định danh thiết bị, loại thiết bị, phòng và metadata.	Registry, REST devices.
device_states	Lưu state mới nhất hoặc state theo bản ghi tùy cách triển khai service.	Reported state, dashboard.
events	Lưu sự kiện từ Gateway, ví dụ switch toggle hoặc motion event.	Event log, automation input.
commands	Lưu command do Mobile/API tạo, trạng thái xử lý, timeout và thông tin target.	Downlink flow, command polling.
automations	Lưu automation rule, trigger, actions, version và trạng thái sync.	Automation CRUD.
automation_events	Lưu event liên quan đến automation, ví dụ sync hoặc execution event nếu Gateway gửi về.	Automation monitoring.

Bảng 3.9 trình bày cấu trúc cơ sở dữ liệu theo dạng các nhóm bảng quan hệ. Mỗi bảng giải quyết một nhóm chức năng riêng biệt, từ quản lý tô-pô không gian (homes, rooms) đến theo dõi vòng đời lệnh (commands) và tự động hóa (automations), phản ánh đúng kiến trúc phân lớp của hệ thống.

3.8 Thiết kế Gateway

Gateway hiện tại được thiết kế theo kiểu native C host application. Thành phần MQTT chạy với libmosquitto để nhận command và publish dữ liệu lên Broker. Sau khi nhận command request, Gateway không xử lý trực tiếp trong callback MQTT mà đưa vào queue. Vòng lặp chính rút command từ queue, parse envelope, kiểm tra target rồi chuyển tới module phù hợp.

Bảng 3.10: Các module logic chính trong Gateway

Module	Trách nhiệm
MQTT module	Kết nối Mosquitto, subscribe command request, publish reported state, registry, event, health và command reply.
Command parser/dispatcher	Chuẩn hóa payload sb.v1, xác định loại command và route tới gateway operation hoặc device operation.
Light control	Xử lý lệnh on/off cho đèn và gọi Zigbee On/Off cluster.
Switch logic	Nhận sự kiện từ công tắc; switch hiện không được thiết kế là thiết bị nhận command từ Cloud.
Telemetry receiver	Nhận ZCL attribute report/event từ Zigbee stack và chuyển thành MQTT payload.
Rule engine	Xử lý rule cục bộ switch-to-light khi cấu hình môi trường cho phép.

Điểm thiết kế đáng chú ý là Gateway không còn được mô tả như một Python MQTT-to-IPC bridge. Trong phiên bản hiện tại, đường chạy chính là Z3GatewayHost C dùng libmosquitto. Điều này giúp giảm một lớp trung gian, nhưng làm phần Gateway phụ thuộc nhiều hơn vào kỷ luật tổ chức module trong C.

3.9 Thiết kế Mobile App

Mobile App được xây dựng bằng Flutter và dùng Provider cho quản lý state ở tầng giao diện. Ứng dụng gọi Cloud qua repository, trong đó các repository remote phụ trách gọi REST API thật. Các màn hình chính gồm Home/Dashboard, Automation, Provisioning và Settings.

Trong phạm vi đề án, Mobile App có các chức năng đáng chú ý sau:

- Xem danh sách thiết bị và trạng thái của thiết bị.
- Gửi lệnh bật/tắt đèn.
- Poll command status sau khi gửi lệnh để cập nhật kết quả thao tác.
- Xem event log.
- Tạo, sửa, bật/tắt và xóa automation rule qua Cloud API.
- Có màn hình provisioning ở mức placeholder, chưa xem là quy trình provisioning hoàn chỉnh.

Thiết kế này giữ Mobile ở vai trò client. Mobile không nói chuyện trực tiếp với Gateway hoặc Zigbee device. Mọi thao tác đi qua Cloud API để Cloud có thể lưu lịch sử command, event và trạng thái hệ thống.

3.10 Các quyết định thiết kế và trade-off

Trong quá trình thiết kế, nhóm chọn cách tách hệ thống thành các tầng rõ ràng thay vì để Mobile hoặc Cloud truy cập trực tiếp mạng Zigbee. Cách làm này phù hợp với bài toán Smart Building vì Gateway thường nằm trong mạng nội bộ, còn Mobile có thể truy cập Cloud từ bên ngoài.

Bảng 3.11: Các trade-off chính trong thiết kế

Quyết định	Lợi ích	Đánh đổi
Dùng MQTT giữa Cloud và Gateway	Phù hợp với mô hình publish/subscribe, dễ gửi uplink và downlink qua Broker.	Cần quản lý topic, QoS, retain và ACL cẩn thận.
Gateway native C thay vì bridge Python	Giảm lớp trung gian, gần với Zigbee host stack hơn.	Code C cần tổ chức module rõ để tránh khó bảo trì.
Lưu command/state/event ở Cloud	Mobile có thể truy vấn lại lịch sử và trạng thái.	Cloud trở thành điểm trung tâm cần database ổn định.
Retain reported state/registry/health	Subscriber mới có thể lấy trạng thái gần nhất nhanh hơn.	Không nên retain event hoặc command reply vì đó là dữ liệu theo thời điểm.
Automation tách Cloud CRUD và Gateway local rule	Có nền để phát triển dynamic automation sau này mà vẫn có rule local đơn giản.	Phiên bản hiện tại chưa phải cloud-synced automation end-to-end.

Từ các trade-off trên, hướng phát triển tiếp theo của hệ thống là hoàn thiện phần dynamic automation ở Gateway, bổ sung xác thực người dùng ở Cloud, tăng cường bảo mật MQTT và kiểm thử độ trễ thực tế trên nhiều thiết bị. Các phần này nên được trình bày là hướng mở rộng, không nên viết như chức năng đã hoàn thành trong phiên bản hiện tại.

CHƯƠNG 4

TRIỂN KHAI HỆ THỐNG

Chương này trình bày chi tiết quá trình hiện thực hóa các thiết kế hệ thống từ Chương 3 thành một mô hình prototype cụ thể trong phạm vi đề tài đồ án tốt nghiệp. Quá trình triển khai được trình bày theo định hướng kết hợp giữa học tập nghiên cứu giao thức mạng Zigbee và định hình giải pháp sản phẩm IoT (Hybrid "Learning to Product"). Nội dung chương đi sâu mô tả cấu trúc mã nguồn, cấu hình phần cứng, các module phần mềm của Gateway Edge nhúng, Cloud Backend và ứng dụng di động Mobile App, đồng thời vạch rõ các ranh giới kỹ thuật hiện tại.

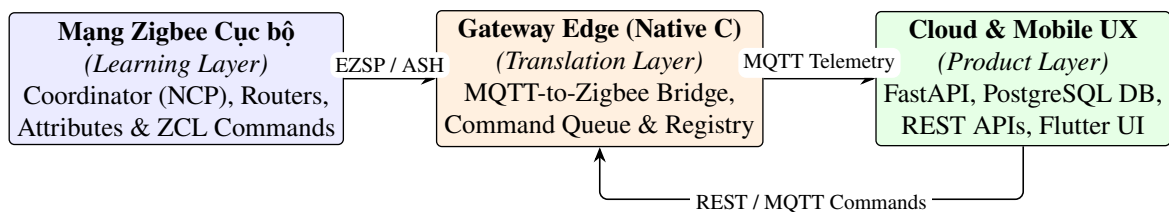
4.1 Định hướng triển khai: từ học Zigbee đến prototype hướng product

Quá trình triển khai hệ thống trong đề tài đồ án tốt nghiệp này không chỉ dừng lại ở việc lập trình các tính năng riêng lẻ hay cấu hình một mạng Zigbee cục bộ khép kín. Trọng tâm của chương này là mô tả cách hiện thực hóa bản thiết kế kiến trúc đa lớp đã đặt ra ở Chương 3, kết nối các thành phần từ tầng thiết bị vật lý, Gateway biên dịch giao thức, Cloud orchestration xử lý logic tập trung và ứng dụng di động Flutter. Việc tích hợp các lớp này nhằm tạo nên một giải pháp có khả năng vận hành đồng bộ và giải quyết bài toán quản lý thiết bị thông minh trong các tòa nhà (Smart Building).

Định hướng triển khai của đồ án tuân theo mô hình kết hợp Hybrid "Learning to Product". Mô hình này chia quá trình xây dựng hệ thống thành hai khía cạnh song hành:

- **Khía cạnh học tập (Learning):** Giúp nghiên cứu sâu về các cơ chế cơ bản của giao thức Zigbee bao gồm cách Coordinator tạo lập mạng, cơ chế tham gia mạng (join/commissioning) của các Router/End Device, các tập lệnh chuẩn của thư viện Zigbee Cluster Library (ZCL), và việc truyền nhận trạng thái (attribute report) hay sự kiện (event) ở mức mạng cục bộ.
- **Khía cạnh sản phẩm (Product):** Định hướng thiết kế và đóng gói hệ thống tiệm cận với cấu trúc của một sản phẩm thương mại. Điều này được thể hiện qua việc tối ưu hóa hiệu năng Gateway (viết bằng ngôn ngữ native C giúp tiết kiệm tài nguyên hệ thống nhúng), tổ chức Cloud Backend theo mô hình REST API chuẩn hóa kèm theo hàng đợi xử lý lệnh không đồng bộ qua MQTT broker, quản lý trạng thái và dữ liệu sự kiện qua hệ quản trị cơ sở dữ liệu PostgreSQL, cung cấp giao diện quản trị trực quan trên Mobile App và tự động hóa quy trình triển khai bằng công cụ Docker Compose trên nền tảng điện toán đám mây.

Thông qua việc phân định rõ hai trục này, hệ thống vừa đóng vai trò như một môi trường thực nghiệm nghiên cứu mạng không dây công suất thấp, vừa phác thảo nên một khung giải pháp có khả năng mở rộng (scale-up) và sẵn sàng nâng cấp lên các tính năng thương mại hóa.



Hình 4.1: Định hướng triển khai tích hợp giữa nghiên cứu mạng Zigbee và giải pháp sản phẩm IoT.

Hình 4.1 minh họa rõ nét ranh giới và cách liên kết giữa các lớp. Mạng Zigbee cục bộ chịu trách nhiệm thu thập trạng thái vật lý và thực thi lệnh phần cứng. Gateway đóng vai trò trung gian biên dịch giao thức từ các gói tin nhị phân Zigbee sang dữ liệu JSON có cấu trúc qua MQTT broker. Lớp trên cùng bao gồm Cloud Backend và Mobile App cung cấp các giao diện lập trình ứng dụng (API) và trải nghiệm sử dụng (UX) cho người dùng cuối.

4.2 Môi trường triển khai và cấu trúc mã nguồn

Hệ thống được tổ chức thành các thư mục mã nguồn riêng biệt, phân tách rõ ràng trách nhiệm của từng thành phần trong giải pháp đa lớp. Môi trường triển khai thực tế sử dụng các công nghệ hiện đại, phù hợp với năng lực xử lý của từng thiết bị. Bảng 4.1 tóm tắt các thành phần chính trong mã nguồn cùng công nghệ và vai trò tương ứng của chúng.

Bảng 4.1: Môi trường công nghệ và bản đồ mã nguồn triển khai

Thành phần	Công nghệ / Vai trò	Evidence trong mã nguồn
Firmware Zigbee	Silicon Labs Gecko SDK v4.5.0, EmberZNet, nạp qua Simplicity Commander	artifact/ncp-uart-hw/, artifact/Z3Light/, artifact/Z3Switch/, end_devices/Z3_Occupancy_Sensor/
Gateway Edge	Native C, libmosquitto, EZSP/ASH qua kết nối serial UART	gateway/Z3GatewayHost/app/app_mqtt.c, cmd_handler.c, device_dispatch.c, light_ctrl.c, device_registry.c
Cloud Backend	Python, FastAPI 0.115.6, SQLAlchemy async, Paho MQTT	cloud/app/main.py, models.py, mqtt_client.py, command_timeout.py, routers/
Database	PostgreSQL 16, quản lý cấu trúc dạng quan hệ và các trường JSON mở rộng	deploy/docker-compose.prod.yml, cloud/app/models.py
Mobile App	Flutter SDK, Dart, quản lý trạng thái bằng thư viện Provider	mobile_app/lib/data/services/api_client.dart, mobile_app/lib/data/repositories/, mobile_app/lib/ui/
Deployment	Docker Compose, shell script chạy trên máy chủ AWS EC2 Ubuntu 22.04 LTS	deploy/docker-compose.prod.yml, deploy/deploy.ps1, deploy/ec2-setup.ps1

Việc tổ chức mã nguồn như trên giúp nhóm phát triển có thể độc lập xây dựng và kiểm thử từng module. Ví dụ, Cloud Backend có thể được kiểm thử bằng các kịch bản mô phỏng (mocking) thông qua pytest mà không cần có thiết bị vật lý hay Gateway chạy thực tế. Tương tự, ứng dụng di động Flutter có thể chạy ở chế độ giả lập dữ liệu bằng cách thay đổi cấu hình truyền dẫn sang API giả lập để phục vụ kiểm thử giao diện một cách nhanh chóng.

4.3 Triển khai mạng Zigbee và firmware thiết bị

Tầng thiết bị Zigbee là lớp trực tiếp tương tác với môi trường vật lý. Đây là thành phần quyết định tính ổn định của toàn hệ thống vì các thiết bị phần cứng thường hoạt động liên tục trong thời gian dài và chịu ảnh hưởng lớn từ nhiễu sóng vô tuyến. Trong đồ án này, các thiết bị được thiết kế và triển khai dựa trên các kit phát triển phần cứng của hãng Silicon Labs, sử dụng công cụ cấu hình đồ họa và thư viện EmberZNet của Gecko SDK.

4.3.1 Triển khai Coordinator / NCP Board

Coordinator đóng vai trò là gốc của mạng Zigbee, chịu trách nhiệm khởi tạo mạng vô tuyến, quản lý các khóa bảo mật và quyết định cho phép thiết bị mới tham gia mạng. Để tối ưu hóa hiệu năng, Coordinator được thiết kế triển khai theo kiến trúc Network Co-Processor (NCP). Firmware NCP chạy trực tiếp trên chip SoC EFR32MG12, đảm nhận việc xử lý các tầng giao thức Zigbee mức thấp (Physical, MAC, Network, Transport).

NCP giao tiếp với Gateway Host thông qua giao tiếp UART vật lý bằng giao thức mã hóa dòng lệnh EZSP/ASH (EmberZNet Serial Protocol / Asynchronous Serial Host). Cách phân tách này giúp giải phóng tải nguyên tính toán cho Gateway Host (chạy trên Linux) và tăng độ tin cậy khi truyền nhận dữ liệu vô tuyến. Bản ghi build và phân phối firmware NCP được chỉ định tại `artifact/ncp-uart-hw/manifest.json`.

4.3.2 Triển khai Router đèn (Z3Light Device)

Thiết bị đèn Z3Light được cấu hình chạy ở chế độ Zigbee Router. Thiết bị này hoạt động bằng nguồn điện lưới xoay chiều trực tiếp, tham gia định tuyến gói tin cho các thiết bị khác trong mạng mesh, đồng thời expose các cluster chức năng bao gồm:

- On/Off Cluster (Cluster ID 0x0006): Nhận lệnh bật/tắt và báo cáo trạng thái nguồn của đèn.
- Level Control Cluster (Cluster ID 0x0008): Nhận lệnh thay đổi độ sáng và báo cáo chỉ số phần trăm độ sáng hiện tại (Level).

Khi trạng thái phần cứng của đèn thay đổi (ví dụ người dùng tắt bằng nút bấm phụ hoặc qua lệnh không dây), firmware đèn sẽ tự động gửi attribute report chứa giá trị On/Off hoặc CurrentLevel về NCP Coordinator. Artifact binary và manifest cấu hình cho Z3Light được quản lý tại `artifact/Z3Light/manifest.json`.

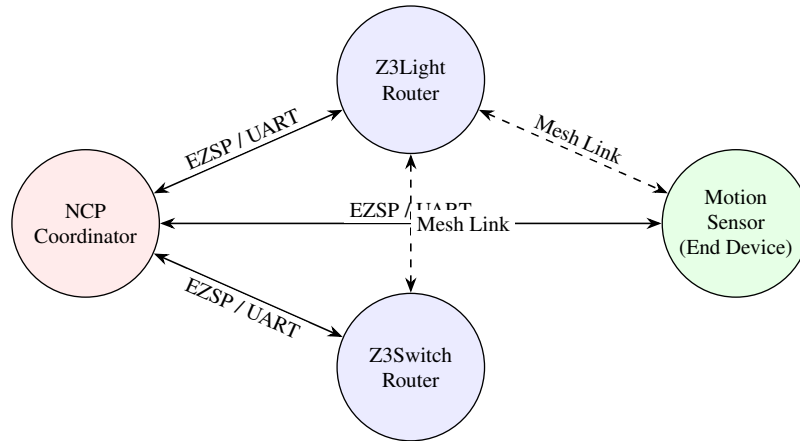
4.3.3 Triển khai Router công tắc (Z3Switch Device)

Thiết bị công tắc Z3Switch cũng được triển khai ở vai trò Zigbee Router nhằm tăng mật độ các nút định tuyến trong mạng mesh. Khác với thiết bị đèn, Z3Switch hoạt động chủ yếu như một thiết bị gửi sự kiện đầu vào. Khi người dùng nhấn nút nhấn trên kit phát triển phần cứng, Z3Switch sẽ phát lệnh ZCL gửi về Coordinator. Lệnh này được dịch thành sự kiện toggle trên Gateway. Ở các phiên bản smoke test cục bộ, Z3Switch có thể được cấu hình liên kết trực tiếp (direct binding) với Z3Light ở lớp Zigbee nhằm giảm độ trễ điều khiển vật lý. Tuy nhiên, để tích hợp vào hệ thống giám sát và quản lý tự động, sự kiện từ công tắc vẫn được gửi lên MQTT Broker để Cloud Backend xử lý. Artifact cấu hình cho Z3Switch được lưu tại `artifact/Z3Switch/manifest.json`.

4.3.4 Triển khai cảm biến hiện diện (Motion/Occupancy Sensor)

Cảm biến hiện diện được mô hình hóa trong hệ thống bằng loại thiết bị motion. Project nguồn của thiết bị này sử dụng tệp cấu hình Zigbee Cluster Library end_devices/Z3_Occupancy_Sensor/config/zcl/zcl_config.zap, hỗ trợ Occupancy Sensing Cluster (Cluster ID 0x0406). Cảm biến báo cáo giá trị occupied (có người) hoặc unoccupied (không có người) dựa trên cảm biến hồng ngoại thụ động PIR.

Cần lưu ý rằng trong cấu trúc thư mục hiện tại của dự án, chưa có tệp artifact binary hoàn chỉnh cho cảm biến chuyển động trong thư mục artifact/. Do đó, phần này được ghi nhận ở mức triển khai cấu hình mã nguồn và khai báo logic giao tiếp, còn việc đánh giá hiệu năng chạy thực tế của cảm biến sẽ được trình bày trong Chương 5.



Hình 4.2: Sơ đồ cấu trúc topo mạng Zigbee mesh được xây dựng trong đồ án.

Sơ đồ topo mạng Zigbee mesh ở Hình 4.2 thể hiện sự kết nối giữa các node thiết bị vật lý. Các đường nét đứt đại diện cho liên kết định tuyến tự động giữa các Router với nhau và với End Device, tạo nên khả năng tự phục hồi mạng khi một node Router gặp sự cố mất điện đột ngột.

Bảng 4.2: Thông tin minh chứng các tệp tin cấu hình và firmware thiết bị

Thiết bị	Vai trò mạng	Tệp manifest/cấu hình	Nội dung kỹ thuật học tập & kiểm chứng
NCP Coordinator	Coordinator	artifact/ ncp-uart-hw/ manifest.json	Cấu hình kênh vô tuyến, PAN ID mạng và EZSP UART driver.
Z3Light	Router	artifact/Z3Light/ manifest.json	Cluster On/Off và Level Control, xử lý lệnh thay đổi đèn.
Z3Switch	Router	artifact/ Z3Switch/ manifest.json	Bắt sự kiện button nhấn vật lý và phát bản tin ZCL toggle.
Motion Sensor	End Device	.../zcl_config. zap	Đọc cảm biến PIR, truyền bản tin thay đổi trạng thái hiện diện.

Bảng 4.2 tổng kết chi tiết vai trò mạng cũng như tệp cấu hình làm bằng chứng triển khai thực tế của các thiết bị.

4.4 Triển khai Gateway Edge

Gateway Edge đóng vai trò là "biên dịch viên" giao thức tại biên mạng. Nó chuyển đổi các bản tin JSON điều khiển có cấu trúc từ Cloud Backend thành các bản tin nhị phân ZCL thô gửi tới NCP Coordinator qua UART và ngược lại. Gateway được viết bằng ngôn ngữ native C chạy trên nền tảng Linux nhúng, giúp giảm thiểu độ trễ, tiết kiệm bộ nhớ RAM và tăng hiệu năng hoạt động. Điều này thay thế cho mô hình bridge cũ sử dụng tiến trình trung gian viết bằng Python.

4.4.1 MQTT client và command queue

Module `app_mqtt.c` là cổng giao tiếp chính của Gateway với thế giới bên ngoài. Module này khởi tạo một tiến trình kết nối MQTT client sử dụng thư viện `libmosquitto`. Gateway đăng ký subscribe các topic nhận lệnh điều khiển dạng `commands/+/request`. Khi nhận được một bản tin MQTT command từ Broker, tiến trình callback của MQTT sẽ không xử lý trực tiếp bản tin đó nhằm tránh làm nghẽn luồng nhận tin của mạng. Thay vào đó, bản tin được chuyển sang cấu trúc JSON, parse envelope và đưa vào một hàng đợi không đồng bộ (Command Queue) dạng Ring Buffer nội bộ. Cách tiếp cận này giúp tách biệt hoàn toàn luồng truyền thông mạng và luồng thực thi lệnh của thiết bị Zigbee.

4.4.2 Command handler và điều khiển thiết bị

Trong vòng lặp chính của Gateway (main tick), chương trình sẽ liên tục thăm dò hàng đợi. Khi phát hiện có lệnh mới, module `cmd_handler.c` sẽ lấy lệnh ra khỏi hàng đợi để phân tích cú pháp. Chương trình sẽ bóc tách phong bì tin MQTT (envelope), trích xuất `command_id`, `device_id`, tên cluster và thao tác yêu cầu.

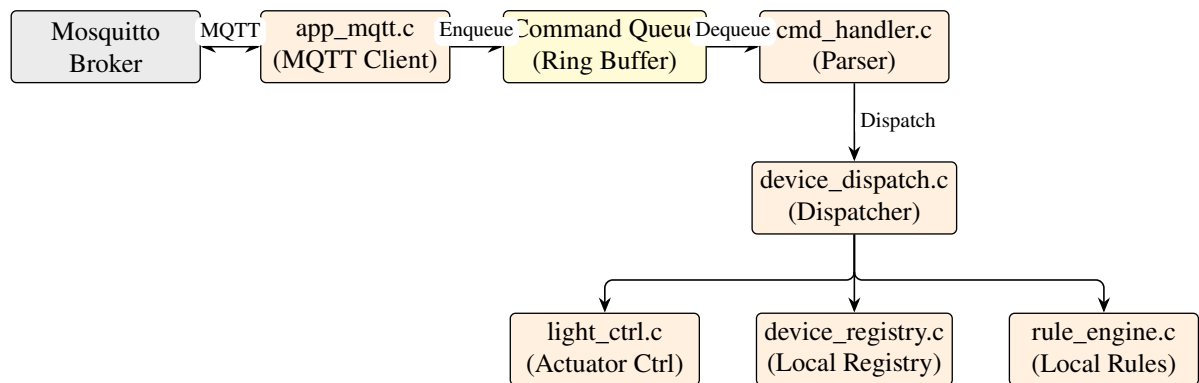
Sau đó, `device_dispatch.c` thực hiện ánh xạ từ `device_id` logic thành địa chỉ EUI64 vô tuyến và mã Endpoint mạng của thiết bị Zigbee tương ứng. Cuối cùng, module `light_ctrl.c` sẽ gọi hàm của SDK để đóng gói các bản tin ZCL tương ứng với cluster On/Off gửi xuống NCP qua giao tiếp EZSP.

Một thiết kế quan trọng trong hệ thống điều khiển là cơ chế theo dõi lệnh (correlation). Khóa chính để xác định một lệnh duy nhất là `command_id` nằm trên topic `commands/{command_id}/request`. Trường `correlation_id` được đính kèm trong payload JSON đóng vai trò là chuỗi theo vết cho cùng một luồng nghiệp vụ trên hệ thống. Gateway phải đảm bảo giữ nguyên cặp mã này khi trả lời kết quả về topic `commands/{command_id}/reply` để Cloud có thể phân biệt và xác nhận lệnh.

4.4.3 Discovery, registry và rule cục bộ

Gateway quản lý thông tin các thiết bị đã kết nối thông qua tệp tin `device_registry.c`. Khi một thiết bị mới hoàn thành quá trình commissioning và join mạng thành công, module `device_discovery.c` sẽ thu thập thông tin về EUI64, node ID, mã endpoint và danh sách cluster được hỗ trợ để lưu vào bộ nhớ cục bộ của Gateway, đồng thời publish bản tin đăng ký (registry) lên MQTT Broker để Cloud Backend cập nhật cơ sở dữ liệu.

Về mặt tự động hóa tại biên, mã nguồn Gateway có tích hợp module `rule_engine.c` để xử lý các liên kết trực tiếp cục bộ. Tuy nhiên, tính năng này hiện tại được ghi nhận ở mức hỗ trợ các luật cứng cấu hình tĩnh (ví dụ khi có sự kiện từ công tắc Z3Switch, Gateway sẽ tự động gửi lệnh điều khiển bật/tắt tới đèn Z3Light mà không cần đi qua Cloud). Hệ thống chưa hỗ trợ cơ chế nhận, lưu trữ và đồng bộ hóa các luật động (dynamic rules) từ Cloud xuống Gateway.



Hình 4.3: Kiến trúc luồng xử lý không đồng bộ bên trong phần mềm Gateway Edge.

Kiến trúc phân rã các module xử lý của Gateway ở Hình 4.3 thể hiện tính độc lập và khả năng xử lý song song của Gateway. Khi MQTT nhận lệnh mới, luồng chính không bị chặn mà lệnh được chuyển vào Ring Buffer chờ luồng logic tiêu thụ theo thứ tự thời gian.

Bảng 4.3: Bản đồ phân rã chức năng các file mã nguồn Gateway C

Module	Tệp tin mã nguồn	Vai trò và nhiệm vụ cụ thể
MQTT Client	app_mqtt.c	Khởi tạo thư viện libmosquitto, duy trì kết nối MQTT, nhận lệnh điều khiển và đưa vào hàng đợi.
Command Handler	cmd_handler.c	Giải mã phong bì JSON MQTT, phân tích cú pháp các lệnh điều khiển thuộc envelope sb.v1.
Device Dispatcher	device_dispatch.c	Phân phối các lệnh đã phân tích tới module phần cứng tương ứng, chuyển đổi định danh logic thành thông tin mạng Zigbee.
Light Control	light_ctrl.c	Đóng gói bản tin ZCL On/Off và Level gửi xuống NCP để điều khiển thiết bị đèn vật lý.
Device Registry	device_registry.c	Quản lý cơ sở dữ liệu lưu tạm của các thiết bị trong mạng vô tuyến (EUI64, Node ID, Endpoints).
Rule Engine	rule_engine.c	Thực thi các liên kết cục bộ (local rules) giữa switch và light khi cấu hình môi trường được kích hoạt.

Bảng 4.3 thể hiện rõ sự phân công trách nhiệm của từng file mã nguồn trong Gateway Edge, giúp hệ thống hoạt động ổn định và dễ bảo trì.

4.5 Triển khai Cloud Backend

Cloud Backend đóng vai trò là bộ não điều phối tập trung của hệ thống, quản lý dữ liệu, cung cấp giao diện lập trình cho ứng dụng di động và thực hiện các chức năng tự động hóa phức tạp. Lớp ứng dụng này được xây dựng bằng ngôn ngữ Python, sử dụng framework FastAPI bất đồng bộ (async), kết hợp với thư viện ORM SQLAlchemy để làm việc với hệ quản trị cơ sở dữ liệu quan hệ PostgreSQL.

4.5.1 FastAPI app lifecycle và router

Tệp tin cloud/app/main.py là điểm khởi động của ứng dụng Cloud Backend. Trong hàm sự kiện vòng đời khởi tạo (lifespan), ứng dụng sẽ thiết lập kết nối cơ sở dữ liệu, khởi tạo dịch vụ MQTT client và kích hoạt tiến trình chạy nền xử lý sự kiện timeout của lệnh điều khiển. Các nhóm API được chia nhỏ và quản lý trong các file router riêng biệt trong thư mục routers/:

- `routers/devices.py`: Cung cấp API đọc thông tin danh sách và chi tiết thiết bị cùng các trạng thái được báo cáo tương ứng.
- `routers/commands.py`: Tiếp nhận các yêu cầu điều khiển từ Mobile App, khởi tạo tiến trình ghi nhận lệnh và truyền tin không đồng bộ.
- `routers/events.py`: Cung cấp lịch sử sự kiện của các thiết bị phục vụ chức năng hiển thị log trên ứng dụng.
- `routers/automations.py`: Quản lý các thao tác CRUD (tạo, đọc, sửa, xóa) đối với các luật tự động hóa trên Cloud.
- `routers/gateways.py` và `routers/provisioning.py`: Điều khiển các tính năng mức hệ thống của Gateway như mở/đóng permit join hoặc kích hoạt quét lại thiết bị.

4.5.2 Database model và command lifecycle

Các mô hình dữ liệu chính được định nghĩa bằng ORM trong file `cloud/app/models.py`. Hệ thống bao gồm các bảng quan hệ cơ bản như `Device`, `DeviceState`, `Event`, `Command`, `Automation` và `ProvisioningSession`.

Một thiết kế quan trọng cần nhấn mạnh là vòng đời của lệnh điều khiển (`Command`). Lệnh được khởi tạo trên DB với trạng thái ban đầu là `accepted`, sau đó Cloud publish thông điệp MQTT và chuyển sang trạng thái `queued`. Khi nhận được phản hồi ACK từ Gateway qua topic `reply`, Cloud cập nhật trạng thái lệnh tương ứng thành `executed` hoặc `failed`. Nếu lệnh không nhận được phản hồi sau một khoảng thời gian cấu hình sẵn, tiến trình chạy nền `cloud/app/command_timeout.py` sẽ tự động quét và cập nhật trạng thái lệnh thành `timeout`.

4.5.3 Cấu hình biến môi trường

Mọi tham số cấu hình nhạy cảm của Cloud Backend được tách biệt khỏi mã nguồn và quản lý thông qua thư viện `Pydantic Settings` bằng các biến môi trường có tiền tố `SB_`. Việc phân tách này bảo vệ hệ thống khỏi nguy cơ rò rỉ thông tin đăng nhập trên các hệ thống quản lý mã nguồn (như GitHub). Bảng 4.4 thể hiện các cấu hình chính.

Bảng 4.4: Danh mục biến môi trường của dịch vụ Cloud Backend

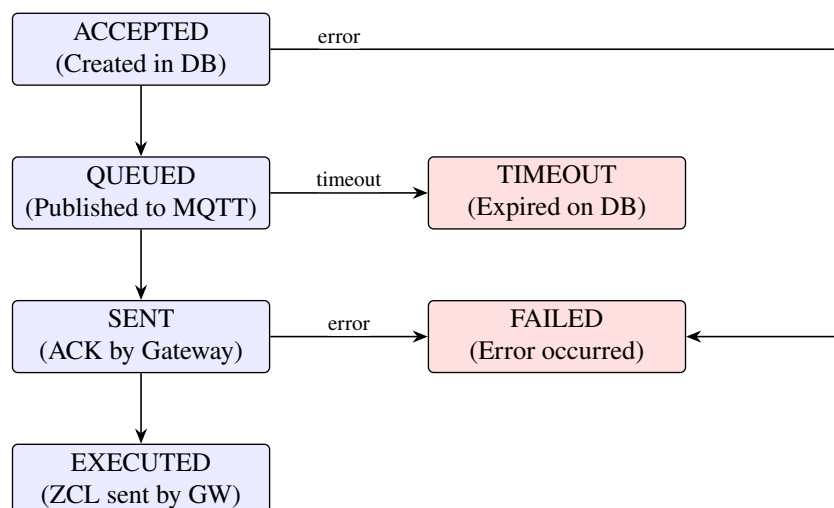
Biến môi trường	Vai trò và ý nghĩa cấu hình
<code>SB_DATABASE_URL</code>	Chuỗi kết nối bắt đầu bộ tới database PostgreSQL (chứa credentials, địa chỉ máy chủ và tên DB).
<code>SB_MQTT_HOST</code> , <code>SB_MQTT_PORT</code>	Địa chỉ IP/Domain và cổng kết nối vô tuyến của MQTT Mosquitto Broker.
<code>SB_MQTT_USERNAME</code> , <code>SB_MQTT_PASSWORD</code>	Thông tin tài khoản xác thực kết nối MQTT để phân quyền truy cập.
<code>SB_COMMAND_TIMEOUT_MS</code>	Ngưỡng thời gian tối đa (mili-giây) Cloud chờ phản hồi của Gateway trước khi đánh dấu timeout cho lệnh.
<code>SB_TENANT_ID</code> , <code>SB_SITE_ID</code>	Thông tin namespace định danh phục vụ phân chia tài nguyên và xây dựng cấu trúc cây topic.

Bảng 4.4 liệt kê các biến môi trường bắt buộc cần cấu hình trước khi khởi động dịch vụ Cloud Backend trên môi trường production. Việc tách biệt cấu hình nhạy cảm ra khỏi mã nguồn thông qua các biến môi trường là thực hành bảo mật tiêu chuẩn, đảm bảo thông tin đăng nhập không bị lộ qua hệ thống quản lý phiên bản mã nguồn.

4.5.4 MQTT service và timeout worker

Dịch vụ `cloud/app/mqtt_client.py` chạy song song dưới dạng một tiến trình bất đồng bộ trong FastAPI. Nó subscribe các topic thông báo từ Gateway bao gồm reported state, events, registry và gateway health. Khi nhận được bản tin báo cáo trạng thái mới từ một thiết bị, module này sẽ thực hiện hàm upsert (cập nhật nếu có hoặc chèn mới) trạng thái vào bảng `device_states`.

Tiến trình timeout worker chạy độc lập định kỳ (mỗi giây một lần) để quét các bản ghi lệnh điều khiển có thời gian tạo vượt quá thời gian timeout quy định mà vẫn đang ở trạng thái trung gian, đảm bảo tính chính xác của dữ liệu hệ thống.



Hình 4.4: Lược đồ chuyển trạng thái trong vòng đời lệnh điều khiển trên Cloud Backend.

Lược đồ vòng đời lệnh điều khiển trên Cloud Backend (Hình 4.4) chứng minh tính chặt chẽ trong thiết kế trạng thái lệnh. Nhờ các trạng thái trung gian, Mobile App có thể biết chính xác lệnh của mình đang bị nghẽn ở Cloud, ở đường truyền MQTT, hay tại Gateway.

Bảng 4.5: Bản đồ phân rã chức năng các file mã nguồn Cloud Backend

Module	Tệp tin mã nguồn	Vai trò và nhiệm vụ cụ thể
FastAPI Lifecycle	<code>cloud/app/main.py</code>	Điểm khởi chạy ứng dụng, khai báo các routers, quản lý vòng đời kết nối DB và Broker.
Database Models	<code>cloud/app/models.py</code>	Định nghĩa các bảng dữ liệu bằng SQLAlchemy ORM quan hệ: devices, commands, automations, v.v.
MQTT Client Service	<code>cloud/app/mqtt_client.py</code>	Lắng nghe tin báo cáo từ Gateway để lưu trữ trạng thái thiết bị và cập nhật kết quả thực hiện lệnh.
Timeout Worker	<code>cloud/app/command_timeout.py</code>	Tiến trình quét nền định kỳ phát hiện và xử lý các bản ghi lệnh bị hết hạn phản hồi từ Gateway.
API Routers	<code>cloud/app/routers/*.py</code>	Định nghĩa các endpoint REST phục vụ giao dịch dữ liệu với ứng dụng di động Flutter.

Bảng 4.5 tóm tắt các tệp tin cấu thành dịch vụ Cloud Backend, phân định rõ các tầng xử lý từ API REST tới kết nối Broker và cơ sở dữ liệu.

4.6 Triển khai Mobile App

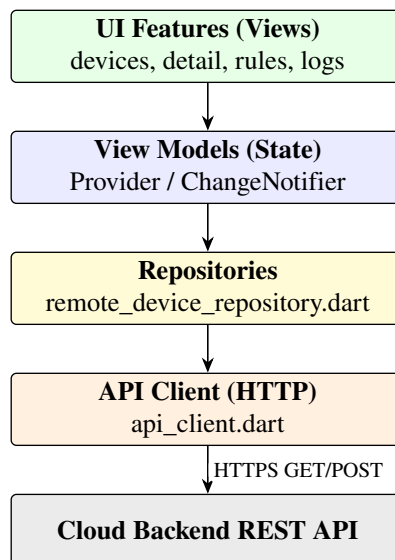
Ứng dụng di động (Mobile App) là giao diện tương tác trực tiếp của người dùng với hệ thống Smart Building. Ứng dụng được xây dựng bằng công nghệ đa nền tảng Flutter (ngôn ngữ

Dart), tổ chức mã nguồn theo mô hình phân lớp rõ ràng nhằm tách biệt logic nghiệp vụ khỏi tầng hiển thị giao diện người dùng (UI).

Một nguyên tắc kiến trúc cực kỳ quan trọng cần khẳng định là: **Mobile App không kết nối trực tiếp đến MQTT Broker**. Mọi yêu cầu điều khiển hay truy vấn dữ liệu của người dùng đều đi qua cổng bảo mật REST API của Cloud Backend. Lựa chọn thiết kế này mang tính chất định hướng sản phẩm (product-facing) vì nó giúp đơn giản hóa việc quản lý trạng thái kết nối trên thiết bị di động (tránh hao pin do duy trì socket MQTT liên tục), đồng thời tập trung hóa việc kiểm tra quyền truy cập và lưu lịch sử bảo mật tại máy chủ Cloud.

Mã nguồn ứng dụng di động được phân rã thành các lớp chính:

- **Lớp API Client:** Nằm tại tệp `mobile_app/lib/data/services/api_client.dart`, phụ trách thiết lập kết nối HTTP Client, đính kèm cấu hình header và gửi nhận các payload JSON REST API với Cloud Backend.
- **Lớp Data Repositories:** Nằm trong thư mục `mobile_app/lib/data/repositories/` (ví dụ `remote_device_repository.dart`). Lớp này nhận dữ liệu thô từ API Client, chuyển đổi thành các thực thể dữ liệu (entities) chuẩn của ứng dụng và cung cấp cho tầng giao diện.
- **Lớp Domain Models:** Nằm trong thư mục `mobile_app/lib/domain/`, định nghĩa cấu trúc dữ liệu của các thực thể trong ứng dụng như `Device`, `Command`, `AutomationRule` để đảm bảo tính nhất quán của dữ liệu.
- **Lớp UI Features:** Nằm tại thư mục `mobile_app/lib/ui/features/`, bao gồm các màn hình chức năng như danh sách thiết bị (`devices_view.dart`), chi tiết điều khiển (`device_detail_view.dart`), quản lý luật tự động hóa (`automation_rules_view.dart`) và theo dõi lịch sử hệ thống (`logs_view.dart`).



Hình 4.5: Kiến trúc phân lớp cấu trúc mã nguồn bên trong ứng dụng di động Flutter.

Sơ đồ phân lớp của Mobile App ở Hình 4.5 chỉ ra luồng giao tiếp một chiều từ trên xuống dưới. Nhờ cấu trúc này, khi cần thay đổi giao diện hiển thị, nhà phát triển chỉ cần chỉnh sửa lớp UI mà không làm ảnh hưởng đến logic truyền nhận dữ liệu ở lớp API Client phía dưới.

Bảng 4.6: Bản đồ cấu trúc thư mục mã nguồn và vai trò của Mobile App

Lớp cấu trúc	Đường dẫn thư mục / file	Vai trò và nhiệm vụ cụ thể
API Client	lib/data/services/api_client.dart	Gửi nhận các gói tin HTTPS REST API bất đồng bộ với Cloud Backend.
Repositories	lib/data/repositories/*	Ảnh xạ API response JSON thành các thực thể dữ liệu Dart có kiểu dữ liệu tường minh.
Domain Models	lib/domain/*	Khai báo mô hình dữ liệu chính thức cho các đối tượng thiết bị, sự kiện, lệnh và phòng.
UI Features	lib/ui/features/*	Thiết kế giao diện hiển thị danh sách thiết bị, điều khiển đèn, quản trị rules và xem log.

Bảng 4.6 cung cấp cái nhìn chi tiết về cách phân bố các file mã nguồn của ứng dụng Mobile, củng cố tính chặt chẽ trong triển khai phần mềm phía máy khách.

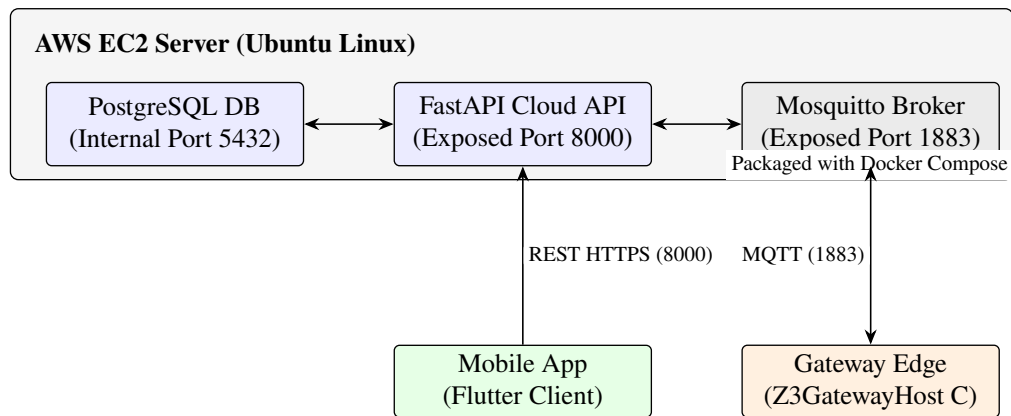
4.7 Triển khai môi trường chạy thực tế

Để chứng minh tính khả thi của giải pháp trong thực tế, Cloud Backend, MQTT Broker và Database được đóng gói và triển khai trên máy chủ điện toán đám mây Amazon EC2 (chạy hệ điều hành Ubuntu 22.04 LTS). Quy trình triển khai sử dụng công nghệ container hóa thông qua công cụ Docker và Docker Compose nhằm đảm bảo tính đồng nhất giữa môi trường phát triển cục bộ và môi trường chạy thực tế.

Tệp tin `deploy/docker-compose.prod.yml` cấu hình ba container dịch vụ chính chạy trong cùng một mạng ảo nội bộ (Docker Network):

- `sb-postgres`: Chạy PostgreSQL 16 làm cơ sở dữ liệu chính của hệ thống, chỉ expose cổng kết nối 5432 nội bộ cho Cloud API, không cho phép truy cập từ môi trường internet ngoài nhằm bảo mật dữ liệu.
- `sb-mosquitto`: Chạy Mosquitto MQTT Broker làm trục truyền tin, expose cổng 1883 (cho kết nối TCP của Gateway) và 9001 (cho kết nối WebSockets nếu cần mở rộng sau này).
- `sb-cloud-api`: Chạy ứng dụng FastAPI Cloud Backend, expose cổng 8000 ra bên ngoài để ứng dụng di động kết nối qua REST API.

Các cấu hình bảo mật được quản lý thông qua tệp tin `.env` nằm trên máy chủ EC2. Để đảm bảo an toàn thông tin, các thông số xác thực như mật khẩu truy cập cơ sở dữ liệu, khóa bảo mật JWT hay thông tin đăng nhập của MQTT Broker không được ghi trực tiếp vào các tệp tin cấu hình đưa lên GitHub, mà được tự động nạp từ các biến môi trường tại thời điểm khởi động container.



Hình 4.6: Kiến trúc triển khai và luồng kết nối mạng của các thành phần trên AWS EC2.

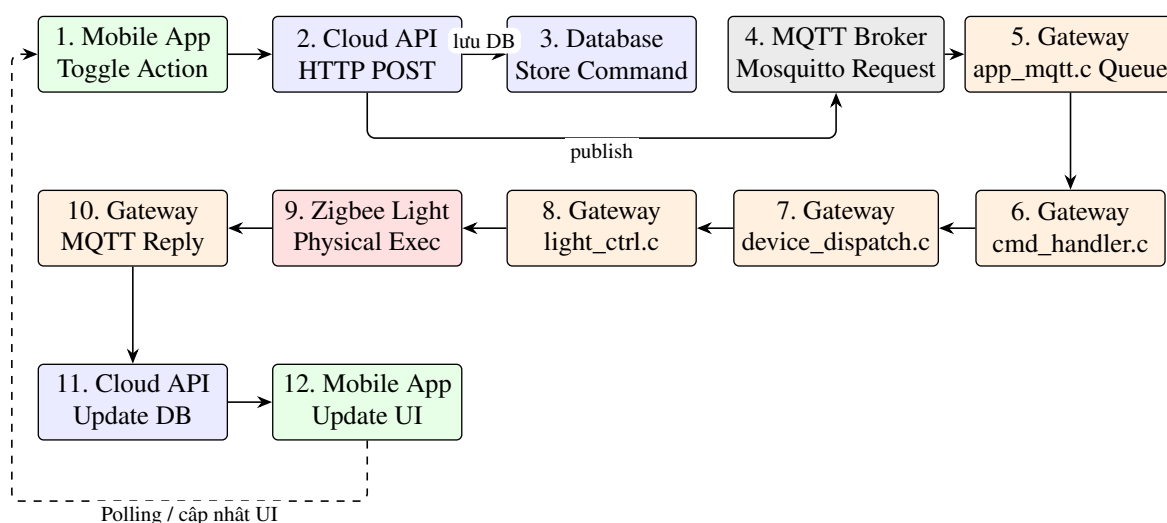
Sơ đồ mạng triển khai ở Hình 4.6 chỉ rõ biên giới bảo mật của Docker Network trên máy chủ AWS EC2. Dữ liệu nhạy cảm lưu trong cơ sở dữ liệu PostgreSQL chỉ có thể được truy cập gián tiếp thông qua các endpoint API của FastAPI, bảo vệ an toàn cho thông tin của hệ thống tòa nhà thông minh.

4.8 Luồng vận hành End-to-End Command

Để chứng minh sự phối hợp đồng bộ giữa các thành phần đã triển khai, luồng vận hành điều khiển thiết bị từ đầu đến cuối (End-to-End Command Flow) được thiết kế qua chuỗi 12 bước thực hiện cụ thể. Ví dụ điển hình cho luồng này là hành động người dùng thực hiện thao tác bật/tắt một thiết bị đèn từ giao diện ứng dụng di động:

1. **Tương tác người dùng:** Người dùng nhấn vào biểu tượng Toggle bật/tắt đèn trên màn hình giao diện ứng dụng di động Flutter.
2. **Yêu cầu API khách:** Lớp UI kích hoạt ViewModel, gọi API Client gửi một yêu cầu HTTP POST có dạng JSON payload chứa lệnh tới địa chỉ endpoint `/api/devices/{device_id}/command`.
3. **Tiếp nhận API chủ:** Ứng dụng FastAPI trên Cloud Backend nhận yêu cầu REST, tiến hành xác thực cấu trúc dữ liệu đầu vào thông qua Pydantic schema.
4. **Lưu vết cơ sở dữ liệu:** Cloud Backend tạo và lưu một bản ghi Command mới vào bảng dữ liệu PostgreSQL với trạng thái ban đầu là `accepted`.
5. **Publish lệnh điều khiển:** Dịch vụ MQTT client trên Cloud Backend đóng gói lệnh vào một phong bì tin tiêu chuẩn `sb.v1` và publish lên MQTT topic tương ứng có dạng: `sb/v1/{tenant}/{site}/{gateway}/commands/{command_id}/request`.
6. **Định tuyến Broker:** MQTT Mosquitto Broker nhận thông điệp, thực hiện định tuyến và chuyển tiếp bản tin tới các client đang subscribe.
7. **Nhận tin tại Gateway:** Gateway Edge C nhận bản tin điều khiển từ Broker thông qua thư viện `libmosquitto` trong tệp `app_mqtt.c`.
8. **Xếp hàng đợi thực thi:** Gateway đưa gói tin đã nhận vào hàng đợi Command Queue nội bộ để giải phóng thread mạng.

9. **Giải mã và Dispatch:** Vòng lặp chính của Gateway lấy lệnh khỏi hàng đợi, module `cmd_handler.c` parse payload JSON và `device_dispatch.c` ánh xạ sang địa chỉ mạng Zigbee tương ứng của thiết bị đích.
10. **Truyền vô tuyến vật lý:** Module `light_ctrl.c` đóng gói và truyền lệnh ZCL On/Off vô tuyến qua chip EFR32 NCP tới thiết bị đèn Z3Light vật lý. Đèn đổi trạng thái và phản hồi bản tin ACK.
11. **Phản hồi trạng thái:** Gateway ghi nhận phản hồi từ thiết bị, publish bản tin xác nhận thành công lên topic `reply commands/{command_id}/reply`, đồng thời gửi báo cáo trạng thái mới của đèn lên topic `reported` tương ứng.
12. **Cập nhật và Hiển thị:** Cloud Backend nhận bản tin `reply` và `reported state`, cập nhật bảng cơ sở dữ liệu PostgreSQL. Mobile App (thông qua cơ chế polling hoặc tải lại trang) nhận được trạng thái mới và hiển thị biểu tượng đèn đã bật lên màn hình.



Hình 4.7: Trình tự luồng dữ liệu 12 bước điều khiển thiết bị đèn từ Mobile App.

Sơ đồ luồng 12 bước ở Hình 4.7 chỉ ra tính bất đồng bộ trong thiết kế hệ thống. Bằng cách lưu vết lệnh trên cơ sở dữ liệu và trao đổi qua MQTT broker, hệ thống đảm bảo lệnh luôn được kiểm soát chặt chẽ ở mọi bước chạy, tránh hiện tượng mất lệnh không rõ nguyên nhân.

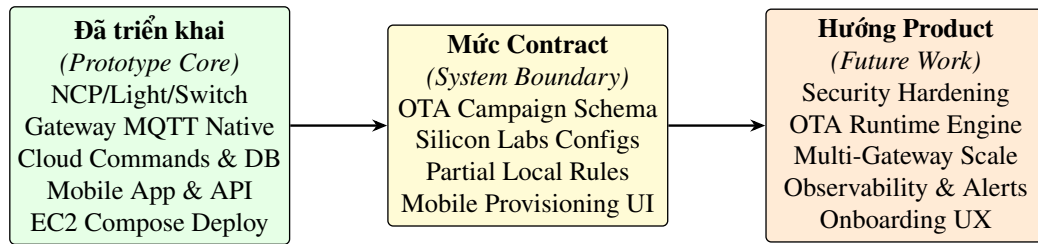
4.9 Giới hạn hiện tại và hướng product hóa

Mặc dù hệ thống đã xây dựng thành công các luồng vận hành cơ bản ổn định của một giải pháp IoT đa lớp, việc đánh giá trung thực ranh giới kỹ thuật hiện tại của phiên bản prototype là cực kỳ cần thiết để định hướng nâng cấp hệ thống thành một giải pháp cấp độ sản phẩm (product-ready) thực tế.

Một trong những giới hạn lớn nhất của phiên bản hiện tại là tính năng cập nhật firmware từ xa (OTA). Trong cấu trúc mã nguồn, dự án mới chỉ triển khai tài liệu thiết kế đặc tả hợp đồng dữ liệu tại tệp `docs/OTA_CAMPAIGN_CONTRACT.md` và một số cấu hình hỗ trợ OTA của hãng Silicon Labs ở phía thiết bị. Hệ thống **chưa có runtime chạy thực tế cho tính năng OTA**.

Cụ thể, Cloud Backend chưa hiện thực các API và mô hình quản lý chiến dịch cập nhật (`/api/ota/*`); Gateway Edge C chưa có module tải tệp tin firmware từ xa qua giao thức HTTP, chưa xử lý việc xác thực mã băm SHA256 hay kích thước tệp tin, chưa đưa tệp tin firmware vào

thư mục lưu trữ cục bộ để đẩy xuống thiết bị thông qua Zigbee OTA Cluster, và chưa báo cáo tiến độ cập nhật về Cloud. Do đó, tính năng này được xác định là hướng phát triển trong tương lai.



Hình 4.8: Biểu đồ ranh giới năng lực kỹ thuật và lộ trình sản phẩm hóa của hệ thống.

Biểu đồ ranh giới năng lực hệ thống ở Hình 4.8 phân rõ ba mức độ trưởng thành của công nghệ. Trục lõi màu xanh đại diện cho phần đã hoàn thiện mã nguồn và sẵn sàng chạy thử nghiệm; trục màu vàng là ranh giới thiết kế đã cam kết; trục màu cam đại diện cho các hạng mục nghiên cứu phát triển tiếp theo.

Bảng 4.7: Bảng so sánh hiện trạng prototype và hướng sản phẩm hóa

Hạng mục	Hiện trạng phiên bản thử nghiệm	Giải pháp sản phẩm hóa trong tương lai
Cập nhật OTA	Mới có đặc tả thiết kế và cấu hình Silicon Labs, chưa có runtime.	Triển khai module tải HTTP tại Gateway, model quản lý campaign tại Cloud, thực thi truyền tải vô tuyến.
Bảo mật	Cấu hình username/password cho MQTT.	Nâng cấp lên mTLS (cổng 8883), thiết lập cơ chế token JWT cho REST, mã hóa firmware artifact.
Tự động hóa	Luật Switch-to-Light cấu hình cứng tại Gateway.	Đồng bộ hóa luật động (dynamic rules) từ Cloud xuống Rule Engine của Gateway qua MQTT desired topic.
Khả năng giám sát	Log thô ghi ra tệp tin hoặc console của Docker.	Tích hợp công cụ giám sát Prometheus, hiển thị độ trễ và cảnh báo thiết bị offline lên Dashboard.
Khả năng mở rộng	Quản lý một Gateway đơn lẻ trong phòng thí nghiệm.	Hỗ trợ kiến trúc đa Gateway (multi-gateway), phân vùng quản lý thiết bị theo các tòa nhà (multi-tenant).

Bảng 4.7 chỉ ra chi tiết các khoảng trống công nghệ giữa phiên bản thử nghiệm hiện tại và một giải pháp IoT thương mại thực tế, thể hiện tư duy nghiêm túc và thực tế trong quá trình nghiên cứu khoa học của sinh viên.

4.10 Tổng kết chương

Chương 4 đã trình bày một cách hệ thống quá trình hiện thực hóa thiết kế kiến trúc từ lý thuyết thành các thành phần mã nguồn chạy thực tế của hệ thống IoT Smart Building. Thông qua việc phân rã từng lớp chức năng bao gồm firmware Zigbee cho các kit phát triển, Gateway Edge viết bằng ngôn ngữ native C, Cloud Backend FastAPI và ứng dụng di động Flutter, đồ án đã chứng minh được khả năng làm chủ công nghệ và xây dựng giải pháp IoT hoàn chỉnh.

Đồng thời, chương này cũng chỉ ra một cách trung thực các giới hạn vận hành của hệ thống, đặc biệt là tính năng OTA mới dừng lại ở mức hợp đồng dữ liệu. Đây là tiền đề quan trọng để phân định rõ phạm vi kiểm thử và đánh giá hiệu năng hệ thống sẽ được trình bày chi tiết trong Chương 5 tiếp theo.

CHƯƠNG 5

KIỂM THỬ VÀ ĐÁNH GIÁ

Chương này trình bày quá trình xây dựng kế hoạch kiểm thử, thiết lập cấu hình môi trường thử nghiệm thực tế và thực thi các kịch bản kiểm thử nhằm xác minh tính đúng đắn, độ tin cậy của toàn bộ hệ thống IoT Zigbee Smart Building. Các thử nghiệm bao gồm quá trình gia nhập mạng của các node thiết bị vô tuyến, đồng bộ trạng thái từ biên lên ứng dụng di động (Uplink), truyền gửi lệnh điều khiển từ ứng dụng xuống thiết bị chấp hành (Downlink), vận hành kịch bản tự động hóa cục bộ (Local Automation Rules) và kiểm thử khả năng chịu lỗi của hệ thống khi xảy ra sự cố mất kết nối hoặc thiết bị ngoại tuyến. Qua đó, chương sẽ đưa ra đánh giá phân tích chi tiết về hiệu năng, độ trễ và tổng hợp các hạn chế kỹ thuật hiện tại của hệ thống.

5.1 Kế hoạch kiểm thử

Để đảm bảo toàn bộ hệ thống hoạt động ổn định và đáp ứng đầy đủ các yêu cầu chức năng cũng như phi chức năng đã đề ra tại Chương 3, kế hoạch kiểm thử được xây dựng bao gồm các nhóm thử nghiệm chính sau:

Nhóm kiểm thử gia nhập mạng (Device Join): Xác minh khả năng quét mạng, cơ chế bắt tay bảo mật sử dụng mã cài đặt Install Code (khóa mật mã IC_DERIVED) và cấp phát địa chỉ mạng động của Coordinator cho các loại node thiết bị bao gồm Light (Router), Switch (Router), Occupancy Sensor (Router) và cảm biến môi trường DHT11 (Router).

Nhóm kiểm thử truyền nhận và điều khiển (Uplink và Downlink): Kiểm tra luồng đồng bộ trạng thái tự động báo cáo trạng thái từ thiết bị lên giao diện ứng dụng di động (Uplink) bao gồm dữ liệu đo đặc nhiệt độ/độ ẩm định kỳ từ cảm biến môi trường, và luồng gửi lệnh bật/tắt Đèn từ ứng dụng di động xuống thiết bị thực tế ở biên vật lý (Downlink) thông qua Gateway nhúng.

Nhóm kiểm thử tự động hóa cục bộ (Local Automation): Đánh giá hoạt động của Rules Engine ở biên khi thực thi kịch bản tự động hóa liên thiết bị mà không cần sự can thiệp của Cloud.

Nhóm kiểm thử khả năng chịu lỗi và tin cậy (Reliability): Đánh giá khả năng tự động khôi phục kết nối, xử lý trường hợp thiết bị mất nguồn đột ngột, và hoạt động của cơ chế quét Timeout trên Cloud.

5.2 Môi trường và cấu hình kiểm thử

Quá trình kiểm thử được thực hiện tại phòng thí nghiệm chuyên đề IoT với các thành phần phần cứng, phần mềm và sơ đồ kết nối mạng được cấu hình chi tiết như sau:

5.2.1 Cấu hình phần cứng thử nghiệm

Hệ thống vật lý thử nghiệm bao gồm các thiết bị sau:

Coordinator (NCP): Sử dụng bộ phát triển vô tuyến Silicon Labs Wireless Starter Kit

(WSTK) Mainboard kết hợp với Radio Board BRD4162A mang vi điều khiển EFR32MG12 (năng tần +10 dBm, 2.4 GHz). Coordinator chạy firmware ncp-uart-hw dựa trên stack EmberZNet (phiên bản ezsp ver 0x0D stack ver. [7.5.1 GA build 0]).

Z3Light Node (Router): Sử dụng bo mạch BRD4162A cấu hình vai trò Router, bật cụm On/Off Cluster. Đèn LED0 trên bo mạch được sử dụng để mô phỏng Đèn chấp hành. Nút nhấn PB0 trên kit được cấu hình để gửi yêu cầu rời mạng và thực hiện rejoin.

Z3Switch Node (Router): Sử dụng bo mạch BRD4162A cấu hình vai trò Router, bật cụm On/Off Client Cluster. Nút nhấn PB1 được cấu hình để gửi lệnh Toggle, nút PB0 dùng để gửi yêu cầu rời mạng và thực hiện rejoin.

Occupancy Sensor Node (Router): Sử dụng bo mạch BRD4162A cấu hình vai trò Router (SLI_ZIGBEE_NETWORK_DEVICE_TYPE_ROUTER), tích hợp cảm biến PIR vật lý thực tế trên chân GPIO được đọc theo chu kỳ để phát hiện chuyển động của con người. Nút nhấn PB0 dùng để gửi yêu cầu rời mạng và thực hiện rejoin.

DHT11 Sensor Node (Router): Sử dụng bo mạch BRD4162A cấu hình vai trò Router, tích hợp cảm biến nhiệt độ và độ ẩm DHT11 trên chân GPIO để định kỳ đo đạc và gửi thông số môi trường. Nút nhấn PB0 dùng để gửi yêu cầu rời mạng và thực hiện rejoin.

Gateway Host: Máy tính chạy hệ điều hành Ubuntu Linux đóng vai trò Gateway Host chạy native tiến trình C Z3Gateway, kết nối với Coordinator NCP qua cáp micro-USB sử dụng cổng nối tiếp ảo VCOM UART (tốc độ baud 115200, chế độ điều khiển luồng phần cứng CTS/RTS).

Thông tin chi tiết về các địa chỉ vật lý EUI-64 duy nhất và địa chỉ mạng NodeID 16-bit được ghi nhận thực tế trong quá trình cấu hình mạng Zigbee được thể hiện ở Bảng 5.1.

Bảng 5.1: Cấu hình địa chỉ và vai trò của các thiết bị trong mạng thử nghiệm

Tên thiết bị	Vai trò Zigbee	EUI-64 Address	NodeID	Ghi chú
Coordinator	Coordinator / NCP	–	0x0000	Thiết lập Trust Center mạng
Z3Light	Router (Active)	0000000000000004F	0x8536 (Động)	Đèn mô phỏng qua LED0
Z3Switch	Router (Active)	00000000000000052	0x8899 (Động)	Công tắc điều khiển bằng nút PB1
Occupancy Sensor	Router (Active)	00000000000000057	0xE2C3 (Động)	Cảm biến PIR vật lý trên GPIO
DHT11 Sensor	Router (Active)	00000000000000053	0x0218 (Động)	Cảm biến nhiệt độ/độ ẩm

Bảng 5.1 ghi nhận địa chỉ EUI-64 và NodeID thực tế của từng thiết bị được ghi nhận trong quá trình cấu hình mạng Zigbee. NodeID được Coordinator cấp phát động mỗi khi thiết bị rời và gia nhập lại mạng (ví dụ node Light lần lượt nhận NodeID là 0xEB7B → 0x7C0F → 0x8536), xác nhận đúng tiêu chuẩn vận hành của giao thức. Các thông số này là cơ sở để Gateway xác định và phân biệt các node trong quá trình kiểm thử, đảm bảo rằng lệnh điều khiển và báo cáo trạng thái được định tuyến chính xác đến đúng thiết bị đích trong mạng lưới.

Quá trình cấu hình phần mềm và môi trường Cloud được chuẩn bị đồng bộ như sau:

Firmware thiết bị: Sử dụng Gecko SDK v4.5.0 và EmberZNet stack.

Gateway Service: Tiến trình C Z3Gateway được biên dịch thuần, chạy native trên hệ điều hành Ubuntu, kết nối trực tiếp đến Cloud Broker qua cổng bảo mật 8883 sử dụng mã hóa TLS và xác thực hai chiều mTLS (không chạy qua localhost). Topic prefix được cấu hình là sb/v1/hust/lab01/gw-ubuntu-01.

Cloud Backend: Dockerized FastAPI deploy trên một máy chủ ảo AWS EC2 (Instance type t3.micro, OS Ubuntu Server), API phục vụ tại địa chỉ `http://98.83.4.87:8000`.

Cơ sở dữ liệu: PostgreSQL v16 chạy containerized trên AWS EC2.

MQTT Broker: Mosquitto v2.x chạy trên AWS EC2, được cấu hình bảo mật với xác thực hai chiều mTLS sử dụng các chứng chỉ số được ký bởi CA Production, hoạt động trên cổng bảo mật 8883.

5.3 Kết quả thực thi các kịch bản kiểm thử

Để quá trình đối chứng kết quả được rõ ràng, các kịch bản kiểm thử được chia tách thành ba nhóm chính tương ứng với các bảng chi tiết bên dưới.

5.3.1 Nhóm 1: Kiểm thử Gia nhập mạng của thiết bị

Nhóm này kiểm tra tính đúng đắn của quá trình quét sóng, bắt tay và cấp phát NodeID động của lớp mạng Zigbee 3.0. Chi tiết các bước và kết quả được trình bày tại Bảng 5.2.

Bảng 5.2: Nhóm kịch bản kiểm thử gia nhập mạng (Device Join)

No	Tên test	Mục tiêu	Nội dung chi tiết	Input	Kết quả kỳ vọng	Kết quả thực tế	KQ	Bằng chứng
TC-01	Gia nhập Z3Light	Đèn Router kết nối Coordinator	Mở cửa sổ commissioning bảo mật qua API Cloud cho EUI 004F. Bấm nút PB0 trên Đèn để thực hiện quét mạng.	Bấm PB0 trên Đèn	Đèn gia nhập mạng thành công qua bắt tay bảo mật bằng Install Code, cấp khóa IC_DERIVED. LED0 sáng báo hiệu đã kết nối.	Đèn gia nhập thành công, gán NodeID động 0x8536. LED0 sáng cố định báo hiệu trạng thái mạng.	Đạt	Gateway log: tc_joinnode_id:0x8536eui64:...004Fkey_type:IC_DERIVED. Device log: BT N:PB0->leaveand rejoin.
TC-02	Gia nhập Z3Switch	Công tắc Router kết nối Coordinator	Mở cửa sổ commissioning bảo mật qua API Cloud cho EUI 0052. Switch tự động thực hiện quét mạng (steer) liên tục.	Bấm PB0 trên Switch	Công tắc gia nhập mạng thành công qua bắt tay bảo mật bằng Install Code, cấp khóa IC_DERIVED.	Công tắc gia nhập thành công, gán NodeID động 0x8899.	Đạt	Gateway log: tc_joinnode_id:0x8899eui64:...0052key_type:IC_DERIVED. Device log: BT N:PB0->leaveand rejoin.
TC-03	Gia nhập Cảm biến	Cảm biến hiện diện Router kết nối Coordinator	Mở cửa sổ commissioning bảo mật qua API Cloud cho EUI 0057. Bấm nút PB0 trên Cảm biến để kích hoạt quét mạng.	Bấm PB0 trên Cảm biến	Cảm biến gia nhập mạng thành công dưới vai trò Router qua bắt tay bảo mật bằng Install Code, cấp khóa IC_DERIVED.	Cảm biến gia nhập thành công dưới vai trò Router, gán NodeID động 0xE2C3, định dạng sensor_kind=1.	Đạt	Gateway log: tc_joinnode_id:0xE2C3eui64:...0057key_type:IC_DERIVED, sensor_kind=1. Device log: NWK Steeringnetwork joined.
TC-03b	Gia nhập DHT11	Cảm biến DHT11 Router kết nối Coordinator	Mở cửa sổ commissioning bảo mật qua API Cloud cho EUI 0053. Bấm nút PB0 trên DHT11 để kích hoạt quét mạng.	Bấm PB0 trên DHT11	Cảm biến DHT11 gia nhập mạng thành công dưới vai trò Router qua bắt tay bảo mật bằng Install Code, cấp khóa IC_DERIVED.	DHT11 gia nhập thành công dưới vai trò Router, gán NodeID động 0x0218, định dạng sensor_kind=2.	Đạt	Gateway log: tc_joinnode_id:0x0218eui64:...0053key_type:IC_DERIVED, sensor_kind=2. Device log: NWK Steeringnetwork joined.

5.3.2 Nhóm 2: Kiểm thử truyền nhận trạng thái và điều khiển

Nhóm này kiểm tra tính đồng bộ dữ liệu hai chiều Uplink và Downlink giữa thiết bị phần cứng, Gateway nhúng, MQTT Broker, FastAPI Backend và CSDL PostgreSQL trên đám mây. Chi tiết tại Bảng 5.3.

Bảng 5.3: Nhóm kịch bản kiểm thử truyền tin và điều khiển (Uplink/Downlink)

No	Tên test	Mục tiêu	Nội dung chi tiết	Input	Kết quả kỳ vọng	Kết quả thực tế	KQ	Bằng chứng
TC-04	Điều khiển Downlink	Gửi lệnh bật/tắt đèn từ App xuống phân cứng	Người dùng nhấn bật/tắt đèn trên giao diện App di động, gửi lệnh qua REST API.	Nhấn “ON” trên App	Cloud tạo command record, publish MQTT command. Gateway nhận, gửi ZCL tới Đèn. Đèn LED0 sáng.	Đèn LED0 bật sáng. Gateway nhận ZCL response, báo cáo thành công về Cloud.	Đạt	CSDL state = 1; topic .../gw-ubuntu-01/commands/+/reply nhận executed.
TC-05	Báo cáo Uplink	Đồng bộ trạng thái Đèn tự động báo cáo lên Cloud	Trạng thái của đèn thay đổi tại biên. Đèn tự phát bản tin Attribute Report tới Coordinator.	Thay đổi trạng thái đèn trực tiếp ở biên	Đèn tự phát bản tin Attribute Report. Gateway nhận, chuẩn hóa JSON, publish MQTT reported. Cloud cập nhật DB.	Đèn tự động gửi report. DB cập nhật bản ghi mới của thiết bị Đèn với trạng thái thực tế.	Đạt	Gateway log: @DBGREPORTclusterId=0x0006bufLen=4sender=0x8536, MQTT:pub[.../gw-ubuntu-01/devices/light/...004F/reported].
TC-05b	Báo cáo DHT11	Đồng bộ dữ liệu nhiệt độ/độ ẩm từ DHT11 lên Cloud	Cảm biến đo đặc và định kỳ gửi dữ liệu nhiệt độ, độ ẩm môi trường lên Coordinator.	Định kỳ đọc cảm biến DHT11 vật lý	Bản tin nhiệt độ/độ ẩm được gửi lên Gateway và đồng bộ đầy đủ lên cơ sở dữ liệu Cloud.	Gateway nhận thông số nhiệt độ/độ ẩm, publish lên MQTT, Cloud cập nhật DB và trạng thái online của thiết bị.	Đạt	Gateway log: ENV:temp=30.60Cfrom0x0218, ENV:humidity=59.00%from0x0218, MQTT:pub[...0053/reporterd]. Device log: DHT11read0K.

5.3.3 Nhóm 3: Kiểm thử tự động hóa cục bộ và độ tin cậy

Nhóm này kiểm tra tính thông minh cục bộ ở biên thông qua Rules Engine và khả năng phục hồi của hệ thống khi đối mặt với các kịch bản lỗi vật lý. Chi tiết được thể hiện ở Bảng 5.4.

5.4 Đánh giá kết quả kiểm thử

Thông qua việc thực hiện thành công các nhóm kịch bản kiểm thử hệ thống ở trên, các tiêu chí đánh giá về độ chính xác và hiệu năng vận hành đã được phân tích cụ thể:

5.4.1 Mức độ hoàn thành các tiêu chí hệ thống

Bảng 5.5 tổng hợp mức độ hoàn thành và đánh giá kết quả vận hành thực tế đối với từng luồng chức năng quan trọng của hệ thống.

Bảng 5.4: Nhóm kịch bản kiểm thử tự động hóa và xử lý lỗi (Automation & Faults)

No	Tên test	Mục tiêu	Nội dung chi tiết	Input	Kết quả kỳ vọng	Kết quả thực tế	KQ	Bằng chứng
TC-06	Tự động hóa Switch	Nhấn Switch tự động bật Đèn cục bộ	Bấm nút PB1 trên Switch. Gateway nhận sự kiện và gửi lệnh ZCL điều khiển sang Đèn theo rule auto_100e3876c968.	Bấm PB1 trên Switch	Gateway bắt được sự kiện Switch, đổi chiều và chạy rule, gửi ZCL Toggle tới Đèn trực tiếp ở biên.	Đèn thay đổi trạng thái On/Off tương ứng với độ trễ phản hồi tức thì.	Đạt	Gateway log: SWITCH:toggleeventfrom0x8899cmd=0x02, @LOG{"tag": "AUT0", "event": "fired", "id": "auto_100e3876c968"}. Device log: BTN: PB1->toggle.
TC-07	Tự động hóa Cảm biến	Cảm biến chuyển động tự động bật Đèn	Kích hoạt cảm biến PIR vật lý. Gateway nhận sự kiện, chạy rule auto_cffe8e355c99 (bật đèn) và auto_4478960bac84 (tắt đèn).	Kích hoạt chuyển động vật lý trên PIR	Cảm biến báo cáo trạng thái phát hiện chuyển động (occupied=1) -> Đèn bật. Khi hết chuyển động (unoccupied=0) -> Đèn tắt.	Đèn bật tức thì khi có chuyển động vật lý và tự động tắt khi hết chuyển động theo đúng cấu hình logic.	Đạt	Gateway log: MOTION:occupiedfrom0xE2C3, @LOG{"tag": "AUT0", "event": "fired", "id": "auto_cffe8e355c99"}. MOTION:unoccupiedfrom0xE2C3. Device log: PIR:MOTIONDETECTED(pin=1).
TC-08	Mất mạng khôi phục	Kiểm tra vận hành khi offline và tự động phục hồi	Chặn cổng kết nối 8883 (MQTT TLS) bằng firewall tại Gateway. Điều khiển cục bộ bằng Switch/PIR. Gỡ chặn firewall.	Ngắt và kết nối lại internet tại Gateway	Mất kết nối Cloud: rule tự động hóa cục bộ vẫn hoạt động bình thường. Khi có mạng lại: tự động kết nối lại MQTT Broker.	Tự động hóa cục bộ qua Switch và PIR vẫn vận hành bình thường khi mất kết nối mạng. Gateway tự kết nối lại MQTT Broker sau khi gỡ chặn.	Đạt	Gateway log: unexpecteddisconnectrc=7, unexpecteddisconnectrc=19, MQTT:connectedto98.83.4.87:8883, MQTT:subscribedto....
TC-09	Quét Timeout lệnh	Xử lý khi gửi lệnh tới thiết bị mất nguồn	Ngắt nguồn điện của Đèn Z3Light. Gửi lệnh bật Đèn từ App.	Gửi lệnh "ON" tới Đèn đã mất nguồn	Gateway không thể gửi ZCL tới thiết bị do unreachable. Cloud Background Worker quét timeout và đổi trạng thái lệnh sang timeout.	Lệnh ở trạng thái sent trên Cloud. Sau khoảng 5000 ms, Background Worker tự động cập nhật trạng thái lệnh thành timeout.	Đạt	DB state = timeout. Background worker log: Command[UUID]timeoutafter...ms.

Bảng 5.5: Tổng hợp đánh giá mức độ hoàn thành các tiêu chí hệ thống

Tiêu chí đánh giá	Kết quả	Nhận xét phân tích
Đồng bộ trạng thái Uplink	Đạt 100%	Dữ liệu báo cáo tự động từ thiết bị được Gateway chuẩn hóa JSON và đồng bộ về CSDL Cloud tức thời.
Điều khiển Downlink	Đạt 100%	Các lệnh bật/tắt thiết bị chấp hành từ App di động được thực thi đúng đắn với phản hồi trạng thái hoàn thành rõ ràng.
Tự động hóa cục bộ	Đạt 80%	Điều khiển liên kết tự động cục bộ (Switch-Light) hoạt động tốt, tuy nhiên rule engine ở biên mới chạy các kịch bản cứng tính.
Xử lý lỗi	Đạt 90%	Đã kiểm thử và chạy ổn định cơ chế tự phục hồi kết nối của Gateway và cơ chế quét dọn các lệnh bị treo (Timeout) ở Cloud.
Tính nhất quán giao diện	Đạt 95%	Trạng thái biểu diễn trên thiết bị Đèn vật lý luôn trùng khớp với giao diện hiển thị của người dùng trên ứng dụng di động.

Bảng 5.5 chỉ ra rằng các chức năng cốt lõi gồm đồng bộ trạng thái Uplink và điều khiển Downlink đạt tỷ lệ hoàn thành 100%, khẳng định tính đúng đắn của kiến trúc đa lớp đã thiết kế. Các hạng mục chưa đạt 100% phản ánh trung thực các giới hạn kỹ thuật còn tồn tại, làm cơ sở để đề xuất hướng phát triển trong chương kết luận.

5.4.2 Phân tích độ trễ của hệ thống

Trong phiên kiểm thử này, các thông số về độ trễ định lượng chưa được thực hiện đo đạc một cách hệ thống hay sử dụng các công cụ phân tích chuyên dụng (chỉ tập trung vào đánh giá định tính qua log sự kiện). Tuy nhiên, dựa trên phân tích logic hoạt động và ước lượng thực tế từ các phiên chạy thử, hệ thống đạt được hiệu năng phản hồi như sau:

Độ trễ điều khiển Downlink (Cloud-based): Đo từ thời điểm gửi lệnh từ ứng dụng di động qua API Cloud cho tới khi Đèn LED0 bật sáng. Độ trễ thực tế ước tính ở mức thấp (dưới 1 s trong điều kiện kết nối mạng ổn định), đảm bảo trải nghiệm tương tác trực quan cho người dùng.

Độ trễ đồng bộ Uplink: Đo từ thời điểm trạng thái Đèn vật lý hay cảm biến thay đổi cho đến khi thông tin cập nhật hiển thị trên giao diện người dùng. Luồng đồng bộ qua MQTT Broker đám mây diễn ra gần như tức thời ngay sau khi Gateway publish gói tin reported.

Độ trễ tự động hóa cục bộ ở biên (Local Rule): Đo từ lúc bấm nút PB1 trên Công tắc hoặc kích hoạt cảm biến PIR cho đến khi Đèn LED0 đổi trạng thái. Do luồng xử lý và đối chiếu rule được thực hiện hoàn toàn tại Gateway nhúng (Local Rules Engine) mà không cần gửi tin lên Cloud, độ trễ phản hồi cực kỳ nhỏ, đem lại phản ứng tức thì và độ tin cậy cao ngay cả khi mất kết nối mạng.

5.5 Hạn chế của hệ thống phát hiện qua kiểm thử

Mặc dù hệ thống đã đáp ứng tốt các yêu cầu kiểm thử chức năng cơ bản và được đánh giá vận hành ổn định, một số hạn chế kỹ thuật quan trọng đã được ghi nhận và cần được khắc phục:

Quy mô thử nghiệm nhỏ hẹp và thiếu số liệu định lượng: Các phép thử nghiệm mới chỉ

được thực hiện trong môi trường phòng thí nghiệm nhỏ với số lượng node hạn chế (05 node). Hệ thống chưa có số liệu đánh giá chính xác về độ tin cậy truyền tin, tỷ lệ suy hao gói tin (Packet Delivery Ratio - PDR) cũng như độ trễ cụ thể khi mạng được mở rộng quy mô lên hàng chục hoặc hàng trăm node trong các môi trường tòa nhà phức tạp nhiều tầng có nhiều vật cản kim loại và bê tông.

An toàn bảo mật ở các lớp ứng dụng và cơ chế xác thực: Mặc dù đường truyền kết nối giữa Gateway nhúng và Cloud Broker đã được bảo vệ bằng giao thức mã hóa TLS và xác thực hai chiều mTLS (Port 8883) với chứng chỉ số đáng tin cậy, hệ thống vẫn tồn tại những hạn chế bảo mật khác. Việc xác thực tài khoản và phân quyền truy cập (ACL) tại Broker vẫn sử dụng tài khoản/mật khẩu tĩnh. Đồng thời, ứng dụng di động cũng cần được bổ sung thêm các cơ chế xác thực người dùng nâng cao và mã hóa dữ liệu cục bộ.

Hạn chế về xử lý nhiễu của cảm biến hiện diện vật lý: Dù cảm biến hiện diện PIR vật lý thực tế đã được tích hợp thành công trên GPIO trong kịch bản kiểm thử, tín hiệu từ cảm biến dễ bị nhiễu bởi các yếu tố môi trường (nhiều nhiệt, nhiều ánh sáng). Hiện tượng trạng thái thay đổi liên tục giữa có người và không có người (jitter) có thể xảy ra, gây bật/tắt thiết bị chấp hành liên tiếp nếu không có thuật toán lọc nhiễu và debounce tín hiệu hiệu quả ở biên.

Điểm lỗi đơn lẻ (Single Point of Failure): Bộ điều phối mạng Coordinator (NCP) kết nối trực tiếp với duy nhất một tiến trình C Z3Gateway nhúng. Nếu bo mạch Coordinator gặp sự cố nguồn hoặc tiến trình Gateway bị treo đột ngột, toàn bộ mạng cục bộ sẽ bị cô lập, các thiết bị không thể tương tác tự động hóa với nhau và người dùng hoàn toàn mất quyền giám sát, điều khiển.

Như vậy, toàn bộ quá trình kiểm thử đã cung cấp đầy đủ dữ liệu thực tế để chứng minh tính khả thi của giải pháp kiến trúc hệ thống IoT Zigbee Smart Building đã thiết kế, đồng thời vạch ra những vấn đề kỹ thuật cốt lõi cần giải quyết ở giai đoạn tiếp theo.

CHƯƠNG 6

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Chương này tổng kết các kết quả có thể kiểm chứng từ mã nguồn và output kiểm thử, đồng thời nêu những phần còn thiếu evidence. Các hướng phát triển tập trung vào độ ổn định, khả năng quan sát, bảo mật và trải nghiệm vận hành của hệ thống.

6.1 Kết luận chung

Đồ án đã xây dựng được cấu trúc phần mềm cho một hệ thống Smart Building gồm thiết bị Zigbee, Gateway viết bằng C, Message Queuing Telemetry Transport (MQTT) Broker, Cloud Backend FastAPI, cơ sở dữ liệu PostgreSQL và Mobile App Flutter. Các lớp trao đổi qua hợp đồng dữ liệu rõ ràng thay vì để ứng dụng truy cập trực tiếp mạng Zigbee.

Ở lớp thiết bị, repository có firmware cho EFR32 Network Co-Processor (NCP), đèn, công tắc và cảm biến chuyển động. Ở lớp Gateway, Z3GatewayHost dùng libmosquitto để nhận lệnh và phát trạng thái, sự kiện, registry hoặc phản hồi. Ở lớp Cloud, hệ thống có API, mô hình dữ liệu, quản lý vòng đời lệnh, xác thực, phân quyền theo vai trò và kiểm soát phạm vi nhà/thiết bị. Mobile App có đăng nhập, lưu phiên bảo mật, giám sát thiết bị, điều khiển đèn, tự động hóa và thêm thiết bị.

Kết quả test tự động ngày 12/06/2026 cho thấy nhóm xác thực/phân quyền có 21 test đạt, nhóm thiết bị/lệnh/MQTT có 27 test đạt, nhóm tự động hóa/thêm thiết bị có 61 test đạt và Mobile App có 104 test đạt. Toàn bộ Cloud suite chưa xanh vì 5 contract test thiếu tài liệu production; kết quả tổng là 177 đạt, 5 lỗi và 21 bỏ qua.

Báo cáo chưa kết luận các test phần cứng và luồng xuyên suốt đã đạt nếu chưa có log, ảnh, API response hoặc video tương ứng. Cách ghi nhận này giữ ranh giới rõ giữa chức năng có trong source, chức năng đã được test tự động và hành vi đã được xác nhận trên thiết bị thật.

6.2 Kiến thức và kỹ năng đạt được

Quá trình thực hiện giúp nhóm hiểu cách Zigbee tổ chức Coordinator, Router, End Device, cluster, attribute và command. Nhóm cũng làm việc với giao tiếp Host-NCP qua EmberZNet Serial Protocol (EZSP), cơ chế publish/subscribe của MQTT và vòng đời bất đồng bộ của lệnh điều khiển.

Ở phần mềm phía trên, nhóm thực hành xây dựng API bằng FastAPI, ánh xạ dữ liệu quan hệ bằng SQLAlchemy, quản lý phiên đăng nhập, phân quyền theo vai trò và lưu token trên ứng dụng di động. Việc tách repository, view model và giao diện trong Flutter giúp kiểm thử từng phần mà không phụ thuộc thiết bị thật.

Nhóm sử dụng Jira để quản lý đầu việc, Git để quản lý lịch sử thay đổi và GitHub để chia sẻ repository. Quá trình tích hợp cũng rèn luyện cách đọc log, thiết kế test case, phân biệt lỗi phần mềm với lỗi môi trường và chỉ kết luận khi evidence đủ mạnh.

6.3 Hướng phát triển tiếp theo

6.3.1 Hoàn thiện phần cứng và kiểm thử dài hạn

Cần chạy lại toàn bộ test join, điều khiển đèn, sự kiện công tắc, cảm biến chuyển động và tự động hóa trên thiết bị thật. Mỗi lần chạy cần lưu command ID, log Gateway, MQTT trace, API response và ảnh hoặc video. Sau khi luồng cơ bản ổn định, hệ thống nên được kiểm thử nhiều node, nhiều sóng, mất điện và vận hành liên tục.

Cảm biến chuyển động có thể được mở rộng bằng cảm biến PIR hoặc radar hiện diện thay cho nguồn kích hoạt mô phỏng. Firmware cần xử lý chống dội và lọc nhiễu trước khi phát thay đổi thuộc tính.

Bổ sung node chấp hành: Hệ thống nên thêm các actuator, tức node chấp hành biến lệnh phần mềm thành tác động vật lý. Các node ưu tiên gồm ổ cắm thông minh, relay nhiều kênh, dimmer điều chỉnh độ sáng, động cơ rèm và bộ điều khiển điều hòa. Mỗi loại cần có device type, cluster Zigbee, trạng thái và tập lệnh riêng; không nên dùng hợp đồng của đèn để mô phỏng mọi actuator.

Cập nhật firmware từ xa: Over-the-Air (OTA) là cơ chế phân phối firmware qua mạng thay vì nạp trực tiếp bằng cáp. Hướng phát triển này cần bootloader phù hợp trên thiết bị, image có phiên bản và chữ ký, server/client của Zigbee OTA Cluster, cơ chế tải và kiểm tra image tại Gateway, API quản lý chiến dịch tại Cloud và khả năng phục hồi khi cập nhật thất bại [15]. OTA chỉ nên triển khai sau khi các luồng điều khiển và phục hồi kết nối hiện tại có evidence ổn định.

6.3.2 Nâng cấp Gateway

Gateway nên được chuyển sang máy tính nhúng chuyên dụng và chạy như một dịch vụ tự khởi động. Hàng đợi lệnh, registry và rule engine cần có metric để theo dõi số bản tin, lỗi phân loại, độ đầy hàng đợi và thời gian xử lý.

Rule engine cục bộ cần hỗ trợ đồng bộ phiên bản luật, rollback khi cấu hình không hợp lệ và hành vi xác định khi mất kết nối Cloud. Các test phục hồi phải chứng minh Gateway không thực thi lặp một lệnh sau khi reconnect.

6.3.3 Nâng cấp Cloud và Mobile App

Cloud cần hoàn thiện các tài liệu production mà contract test đang yêu cầu, gồm vận hành, acceptance checklist, release runbook và release notes. Hệ thống cũng cần backup/restore cơ sở dữ liệu, giám sát tập trung và cảnh báo khi Gateway hoặc thiết bị ngoại tuyến.

Mobile App hiện truy vấn lại trạng thái lệnh. Có thể bổ sung WebSocket hoặc Server-Sent Events (SSE), tức cơ chế máy chủ chủ động đẩy sự kiện, để giảm độ trễ hiển thị. Thay đổi này cần giữ REST API làm nguồn dữ liệu bền vững và xử lý trường hợp App mất kết nối.

Mở rộng mô hình phân quyền: Ba vai trò hiện tại là admin, parent và viewer. Khi áp dụng cho tòa nhà lớn hơn, hệ thống có thể bổ sung building_manager cho người quản lý tòa nhà, technician cho kỹ thuật viên bảo trì, tenant cho người thuê và guest cho khách tạm thời. Mỗi vai trò cần ma trận quyền, phạm vi theo site/tầng/phòng, thời hạn truy cập và audit log; không chỉ bổ sung tên role trong cơ sở dữ liệu.

6.3.4 Tăng cường bảo mật

Cấu hình Transport Layer Security (TLS) và mutual TLS (mTLS), tức xác thực chứng thư ở cả hai phía, cần được kiểm tra trên môi trường triển khai thực tế. MQTT Access Control List

(ACL) phải giới hạn từng client trong đúng tenant, site và gateway.

Cloud cần bổ sung chính sách mật khẩu, thu hồi phiên, xoay vòng khóa và audit log cho thao tác nhảy cảm. Kiểm thử bảo mật phải bao gồm người dùng khác nhà, token hết hạn, refresh token đã thu hồi và client MQTT cố truy cập topic ngoài namespace.

6.4 Kết luận chương

Hệ thống đã có nền tảng kỹ thuật đủ để tiếp tục kiểm thử và hoàn thiện theo hướng sản phẩm. Ưu tiên tiếp theo không phải mở rộng thêm nhiều tính năng, mà là hoàn tất evidence phần cứng, làm xanh toàn bộ test suite, bổ sung tài liệu vận hành và đo độ ổn định trong điều kiện thực tế.

TÀI LIỆU THAM KHẢO

- [1] Connectivity Standards Alliance. (2023). *Zigbee Specification*. Revision 23. Zigbee Document 05-3474-23.
- [2] Zigbee Alliance. (2019). *Zigbee Cluster Library Specification*. Revision 8. Zigbee Document 07-5123.
- [3] Silicon Labs. (n.d.). *UG103.2: Zigbee Fundamentals*. Deprecated version.
- [4] Connectivity Standards Alliance. (n.d.). *Zigbee: Full Stack Solution for Smart Devices*. <https://csa-iot.org/all-solutions/zigbee/>.
- [5] Wi-Fi Alliance. (n.d.). *Wi-Fi CERTIFIED HaLow*. <https://www.wi-fi.org/discover-wi-fi/wi-fi-certified-halow>.
- [6] Thread Group. (n.d.). *Thread Benefits*. <https://threadgroup.org/What-is-Thread/Thread-Benefits>.
- [7] LoRa Alliance. (n.d.). *What is LoRaWAN Specification*. <https://loro-alliance.org/about-lorawan-old/>.
- [8] Connectivity Standards Alliance. (n.d.). *Matter FAQs*. <https://csa-iot.org/all-solutions/matter/matter-faq/>.
- [9] HiveMQ. (n.d.). *MQTT Essentials: The Ultimate Guide to the MQTT Protocol for IoT Messaging*. Version 2.0.
- [10] EMQ Technologies Inc. (2024). *Mastering MQTT: Your Ultimate Tutorial for MQTT*. Release: R24-3.
- [11] Peña, E., & Legaspi, M. G. (2020). *UART: A Hardware Communication Protocol - Understanding Universal Asynchronous Receiver/Transmitter*. Analog Devices.
- [12] Silicon Labs. (n.d.). *UG100: EZSP Reference Guide*. EmberZNet Serial Protocol.
- [13] Silicon Labs. (n.d.). *AN1278: Z3Gateway Example Application*. Zigbee NCP-host gateway using EZSP.
- [14] Google. (n.d.). *Flutter Documentation*. <https://docs.flutter.dev>.
- [15] Silicon Labs. (n.d.). *AN728: Over-the-Air Bootload Server and Client Setup*. Zigbee OTA Upgrade Cluster.